北京开源芯片研究院
BEIJING INSTITUTE OF OPEN SOURCE CHIP

中国开放指令生态（RISC-V）联盟
China RISC-V Alliance

香山开源处理器社区
XiangShan Open-Source Processor Community

XiangShan: An Open-Source High-Performance RISC-V Processor and Infrastructure for Academia and Industry

The XiangShan Team

RISC-V Summit Europe 2025 @ Paris, France

May 12, 2025

Together for a shared future

- ASPLOS'25 @ Rotterdam, Netherlands
- HPCA'25 @ Las Vegas, USA
- MICRO'24 @ Austin, USA
- ASPLOS'24 @ San Diego, USA
- HPCA'24 @ Edinburgh, Scotland
- MICRO'23 @ Toronto, Canada
- ASPLOS'23 @ Vancouver, Cananda

XiangShan Home page

XiangShan Document

- English Biweekly including latest progress and performance data
- Official document continuously updating
- Close connections with open-source community
- Feel free to contact us through email or file issues on GitHub!
  - all@xiangshan.cc
  - https://github.com/OpenXiangShan/XiangShan

What we will cover in this tutorial

- **Introduce of XiangShan (11:30 – 12:00, 30 minutes)**
  - The Era of Open-Source Chip
  - XiangShan Roadmap
  - Agile Development Tools
  - Infrastructure for Research

- **CPU Microarchitecture (12:00 - 12:30, 30 minutes)**
- **Development workflows (12:30 – 13:00, 30 minutes)**

What we will cover in this tutorial

- Introduce of XiangShan (11:30 – 12:00, 30 minutes)

- **CPU Microarchitecture (12:00 - 12:30, 30 minutes)**
  - Frontend: *branch prediction and instruction fetch*
  - Backend: *out-of-order scheduler, execution units*
  - Load/Store Unit: *LSQ, pipelines, TLBs, data caches*
  - L2/L3 caches and *prefetchers*

- Development workflows (12:30 – 13:00, 30 minutes)

What we will cover in this tutorial

- Introduce of XiangShan (11:30 – 12:00, 30 minutes)
- CPU Microarchitecture (12:00 - 12:30, 30 minutes)
- **Development workflows (12:30 – 13:00, 30 minutes)**
  - Introduction of their usages
    - *How to develop on XiangShan with MinJie*
  - Simulation and Debugging
  - Debug Demo

Part I

The Era of Open-Source Chip

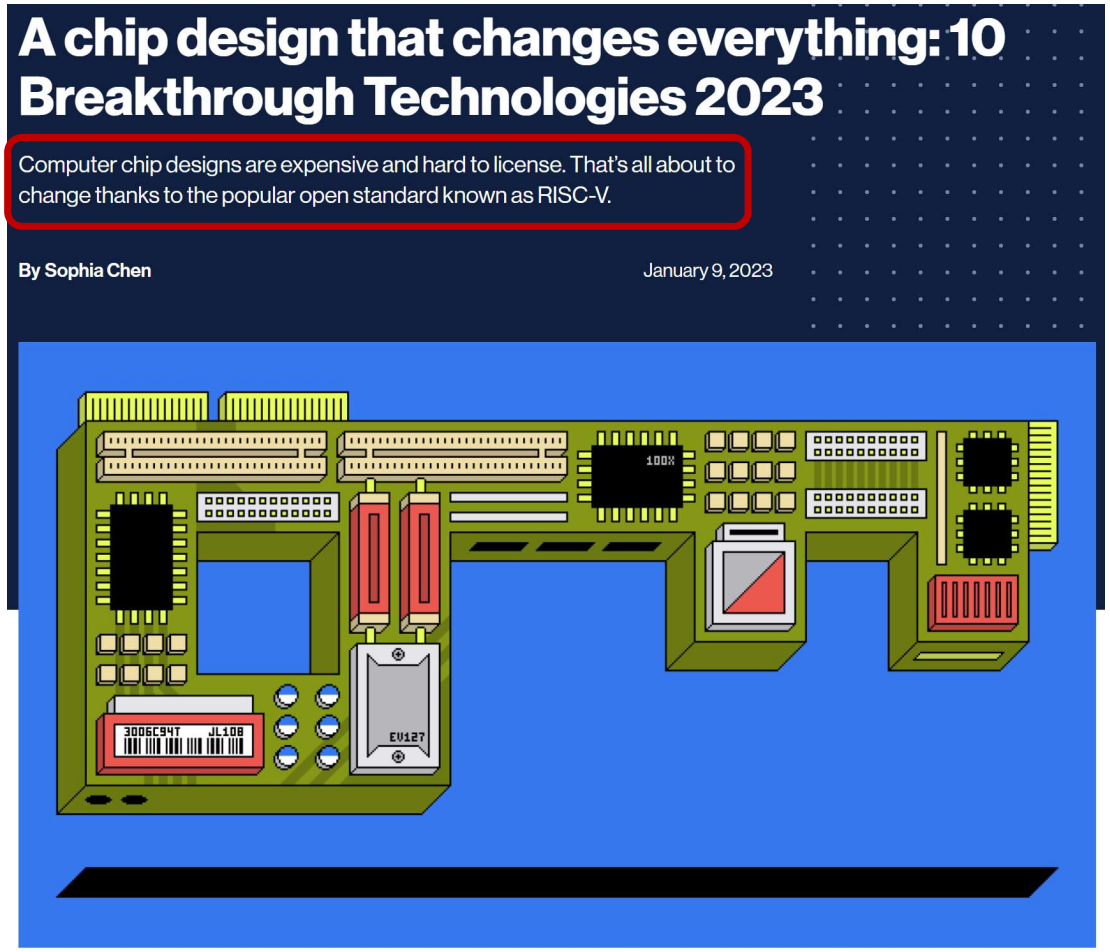
A chip design that changes everything: RISC-V

- **10 Breakthrough Technologies 2023**

Ever wonder how your smartphone connects to your Bluetooth speaker, given they were made by different companies? Well, Bluetooth is an open standard, meaning its design specifications, such as the required frequency and its data encoding protocols, are publicly available. Software and hardware based on open standards—Ethernet, Wi-Fi, PDF—have become household names.

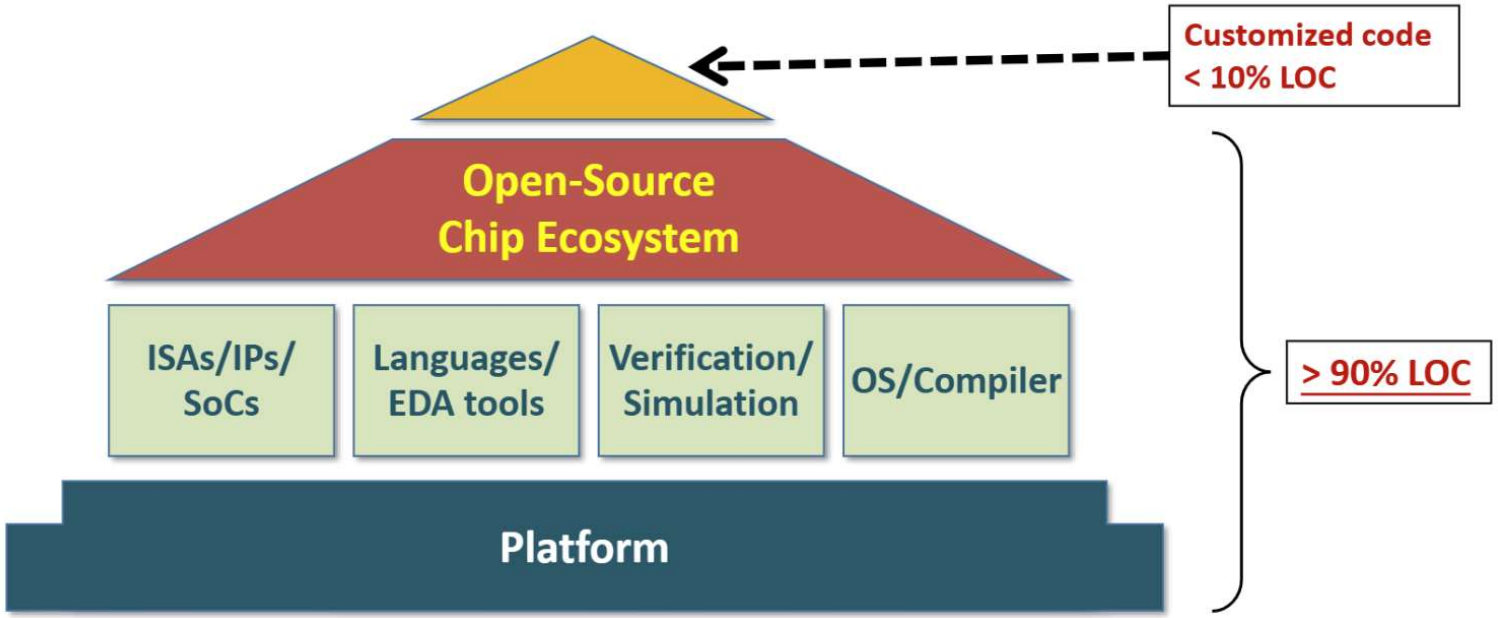
Now an open standard known as RISC-V (pronounced "risk five") could change how companies create computer chips.

--- MIT Technology Review



Open-Source Chip Ecosystem (OSCE)

- **Goal: mirror the success of the open-source software ecosystem**



To Lower the barrier of chip development

By saving the cost of IPs, EDA tools and engineers in chip design

* Yungang Bao, The Four Steps to An Open-Source Chip Design Ecosystem, ACM SIGARCH Visioning Workshop, June 2019

Three levels of OSCE

- **L1: OPEN** ISA
- **L2: OPEN** Design & Implementation
- **L3: OPEN** Tools & Infrastructure

3 Open Tools & Infrastructure

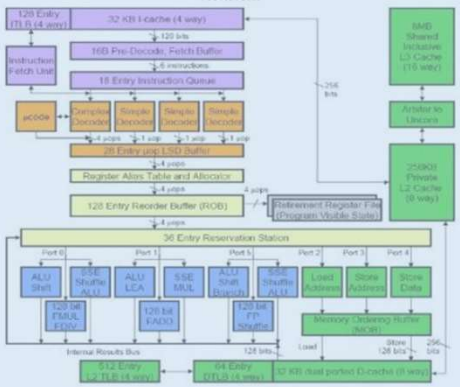
Microarch. → Engineering → EDA Tools

ISA Spec.

RISC-V®

1 Open ISA

Docs



RTL codes

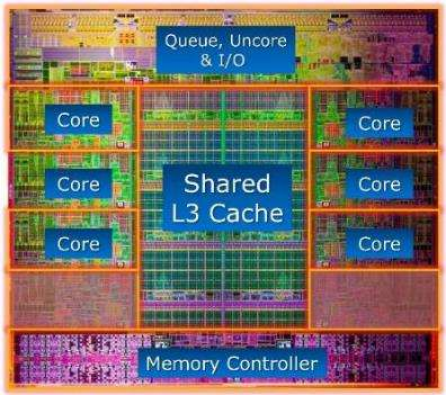
```
component DebugCoreTop is
  port (
    -- Trigger and Data
    cu_Clk      : in    std_logic_vector(2 downto 0) := (others => '0');
    cu0_Trig    : in    t_trig_0 := (others => (others => '0'));
    cu1_Trig    : in    t_trig_1 := (others => (others => '0'));
    cu2_Trig    : in    t_trig_2 := (others => (others => '0'));
    cu0_Data    : in    t_data_0 := (others => (others => '0'));
    cu1_Data    : in    t_data_1 := (others => (others => '0'));
    cu2_Data    : in    t_data_2 := (others => (others => '0'));

    -- Downstream I2C
    SCL         : in    std_logic := '0';
    SDA         : inout std_logic := '0';

    -- Upstream
    gt_RefClk_p : in    std_logic := '0';
    gt_RefClk_n : in    std_logic := '0';
    gt_RX_p     : in    std_logic_vector(2 downto 0) := (others => '0');
    gt_RX_n     : in    std_logic_vector(2 downto 0) := (others => '0');
    gt_TX_p     : out   std_logic_vector(2 downto 0);
    gt_TX_n     : out   std_logic_vector(2 downto 0)
  );
end component;
```

2 Open Design & Implt

Layout



Why do we start XiangShan?

- **Why RISC-V**: Free and open ISA
- **Why high-perf** : Most RISC-V processors are for IoT/edge, but both academic and industrial community need high-performance RISC-V processors
- **Why open-source**: An open and innovative hardware platform
- **Build a leading platform with end-to-end agile development flows and tools**

Linux

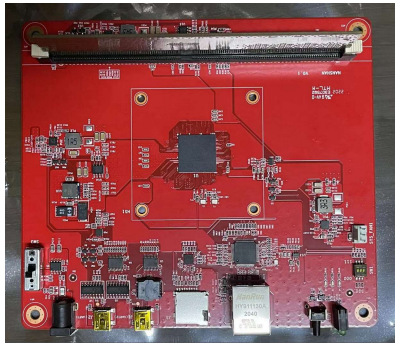
V.S.

XIANGSHAN

Envision:
"Hardware Version of Linux"

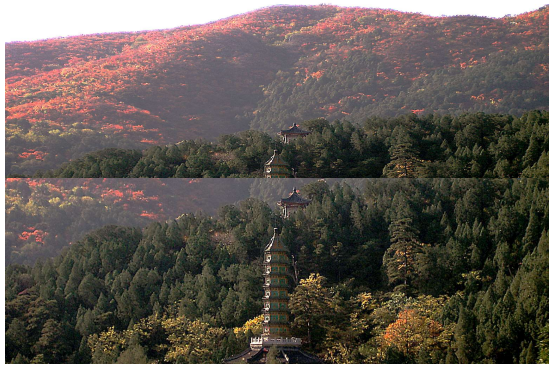
Part II

XiangShan: Open-Source High Performance RISC-V Processor

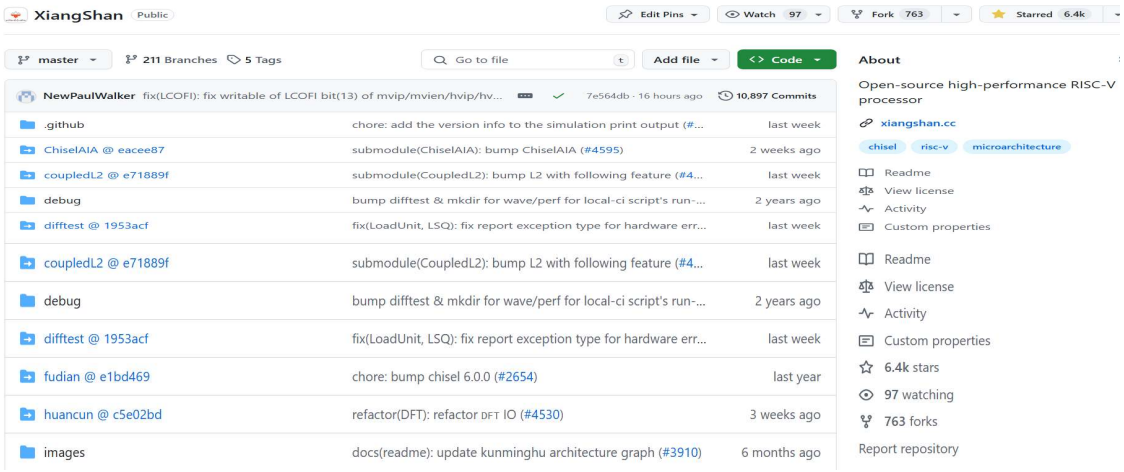


XiangShan: Open-Source High Performance Processors

- **L1: OPEN** ISA
- **L2: OPEN** Design/Implementation
- **L3: OPEN** Framework/Tools



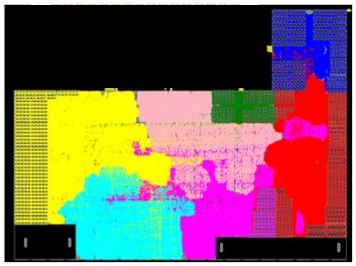
Fragrant Hill in Beijing

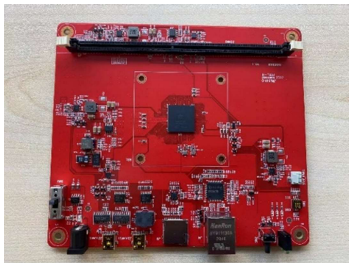


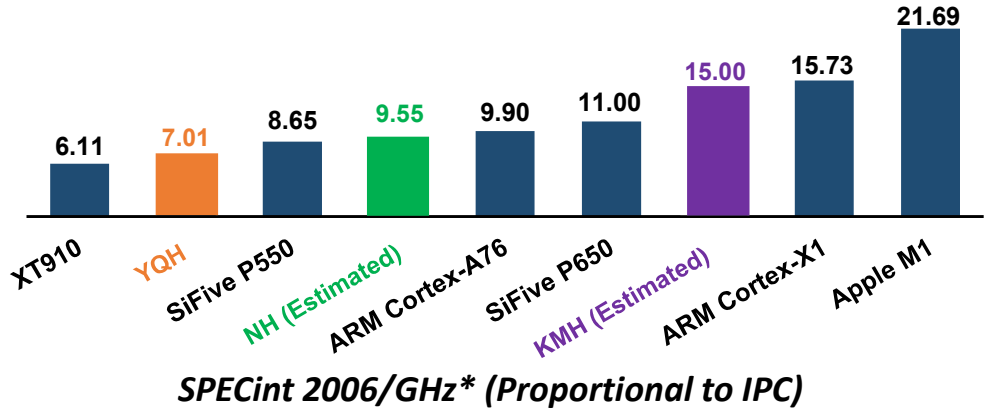
> 6.4K stars, > 760 forks on GitHub

XiangShan: Open-Source High Performance Processors

- 1st generation: Yanqihu (YQH)
  - RV64GC, single-core, superscalar OoO
  - **28nm tape-out, 1.3GHz, July 2021**
  - **SPEC CPU2006 7.01@1GHz, DDR4-1600**
- 2nd generation: Nanhu (NH)
  - RV64GCBK, dual-core, superscalar OoO
  - **14nm GDSII delivery, 2GHz, 2023 Q3**
  - **Estimated SPECint 2006 19.10@2GHz**
- 3rd generation: Kunminghu (KMH)
  - RV64GCBKHV, quad-core, superscalar OoO
  - **Advanced-node, 3GHz, 1.5x IPC of NH**
  - **Close collaboration with industrial partners**







* Source: XT910@ISCA'20, SiFive, AnandTech

XiangShan Gen 1: Yanqihu

- **Test chip developed almost entirely by students**
  - RV64GC, 11-stage, superscalar, out-of-order
  - 5.3 CoreMark/MHz (gcc-9.3.0 –O2)
- **Tape-out**: single XiangShan core with 1MB L2 Cache
  - 28nm, 1.3GHz, July 2021

Yanqi Lake in Beijing

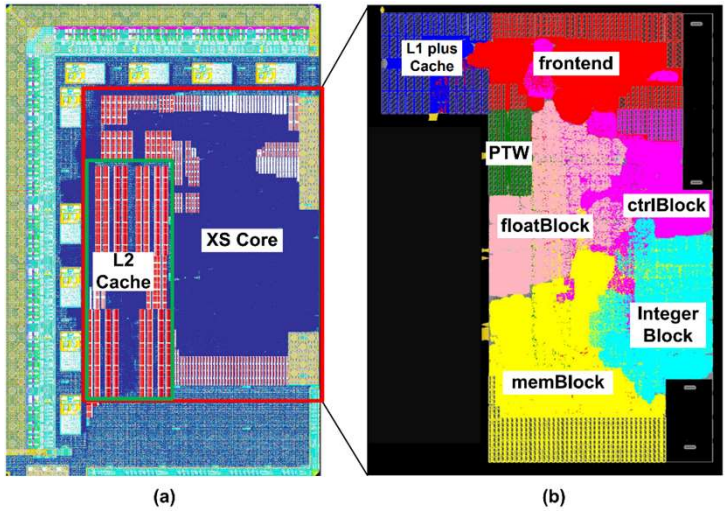
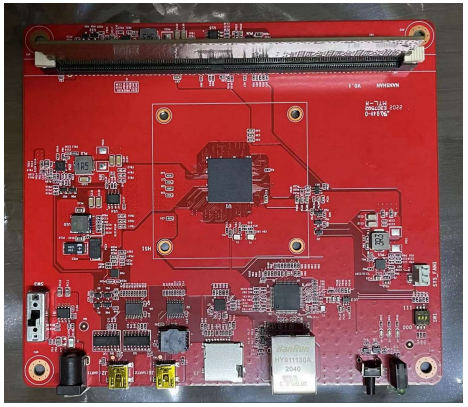


Figure. Layout of (a) the entire chip; (b) the core

Tape-out information for the processor core	
Process Node	28nm
Die Size	8.6 mm^2
Std Cell	5.05M, 4.27 mm^2
Mem	261, 1.7mm^2
Density	66%
Cell	ULVT 1.04%, LVT 19.32%, SVT 25.19%, HVT 53.67%
Estimated Power	5W
Frequency	1.3GHz, TT85C

Real Chip of XiangShan Gen 1 Yanqihu

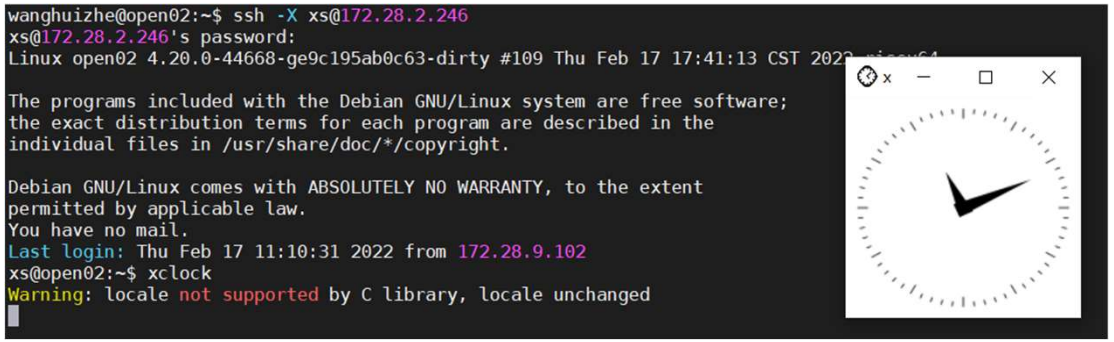
- **The chip was back in January 2022**
  - SoC: CPU, SPI Flash, UART, SD card, Ethernet, DIMM
  - Correctly running Debian with SD card and ethernet
- **Performance: SPEC CPU2006 7.01@1GHz**



SPECint 2006 @ 1GHz	
400.perlbench	6.14
401.bzip2	4.37
403.gcc	6.71
429.mcf	6.83
445.gobmk	7.92
456.hmmer	5.24
458.sjeng	6.85
462.libquantum	17.71
464.h264ref	10.91
471.omnetpp	5.65
473.astar	5.16
483.xalancbmk	7.35

SPECfp 2006 @ 1GHz	
410.bwaves	9.28
416.gamess	6.59
433.milc	8.41
434.zeusmp	7.65
435.gromacs	4.99
436.cactusADM	3.97
437.leslie3d	6.93
444.namd	8.00
447.dealII	10.17
450.soplex	7.03
453.povray	7.14
454.Calculix	2.86
459.GemsFDTD	8.35
465.tonto	6.42
470.lbm	10.39
481.wrf	7.26
482.sphinx3	9.07

SPECint 2006: 7.03@1GHz
SPECfp 2006: 7.00@1GHz



SSH into the Debian on XiangShan, and run a GUI program via X11 forwarding

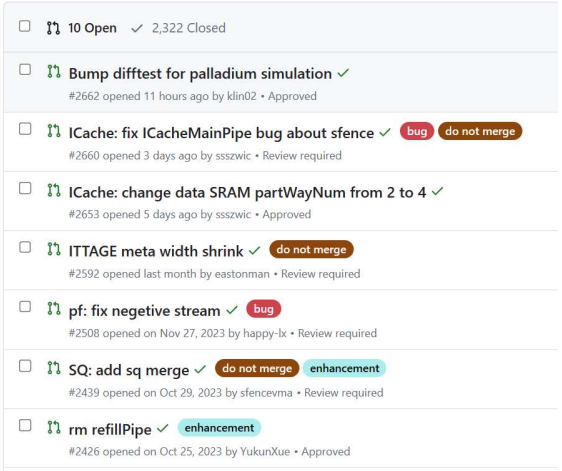
XiangShan Gen 2: Nanhu

- **Target: 2GHz@14nm, SPEC CPU2006 ~20 marks**

- **New frontend**: decoupled BP and instruction fetch
- **Improved backend**: better scheduler, instruction fusions, move elimination, and more
- **New L2/L3 cache**: designed for high frequency and high performance with hybrid prefetchers
- **Verified under dual-core config** (RV64GCBK), more devices support (PCIe, USB, …)

Setup an open and standardized development workflow

A lake in Jiaxing, Zhejiang, China



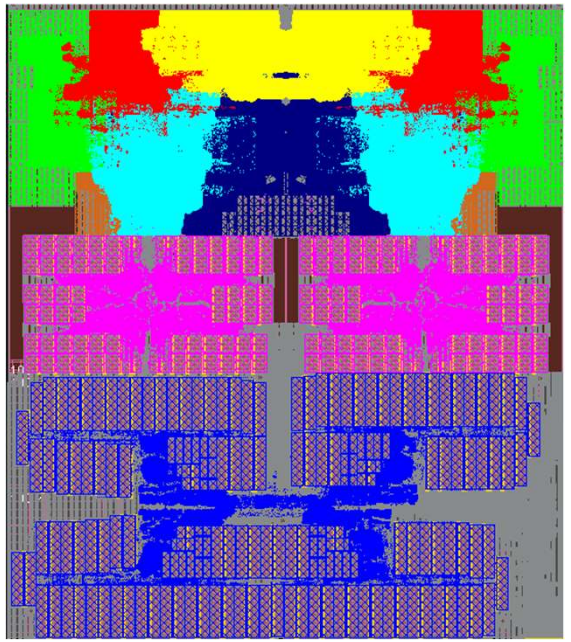
Pull Request Snapshot

Estimated Performance of XiangShan Gen 2 Nanhu

- **Estimated SPECint 2006 19.10, SPECfp 2006 22.18@2GHz**
  - RTL simulation, DDR4-2400 under DRAMsim3
  - Compile with GCC 10.2.0, -O2, RV64GCB

SPECint 06	
400.perlbench	19.27
401.bzip2	11.36
403.gcc	21.97
429.mcf	20.53
445.gobmk	15.97
456.hmmer	19.22
458.sjeng	17.22
462.libquantum	36.99
464.h264ref	28.54
471.omnetpp	14.02
473.astar	14.19
483.xalancbmk	21.48
SPECint2006@2GHz	***19.10***

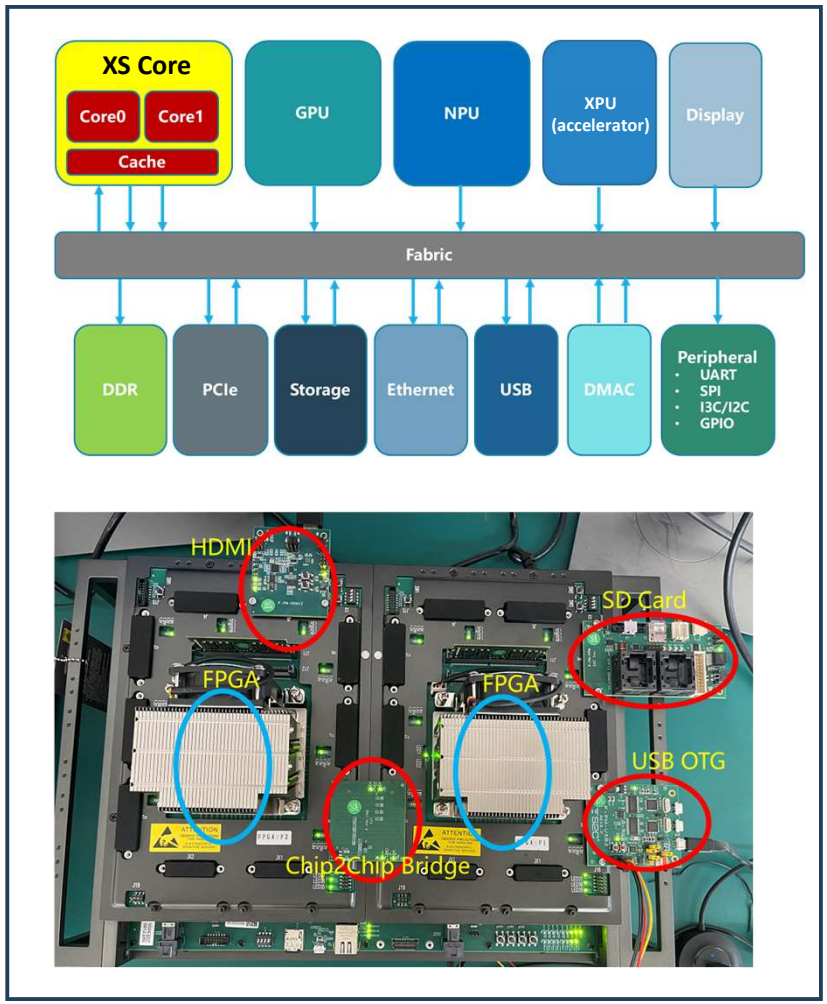
SPECfp 06	
410.bwaves	18.09
416.gamess	23.82
433.milc	18.33
434.zeusmp	28.19
435.gromacs	17.53
436.cactusADM	24.26
437.leslie3d	20.28
444.namd	23.83
447.dealII	33.50
450.soplex	25.61
453.povray	27.06
454.Calculix	9.18
459.GemsFDTD	24.66
465.tonto	17.68
470.lbm	32.04
481.wrf	19.73
482.sphinx3	28.38
SPECfp2006@2GHz	***22.18***

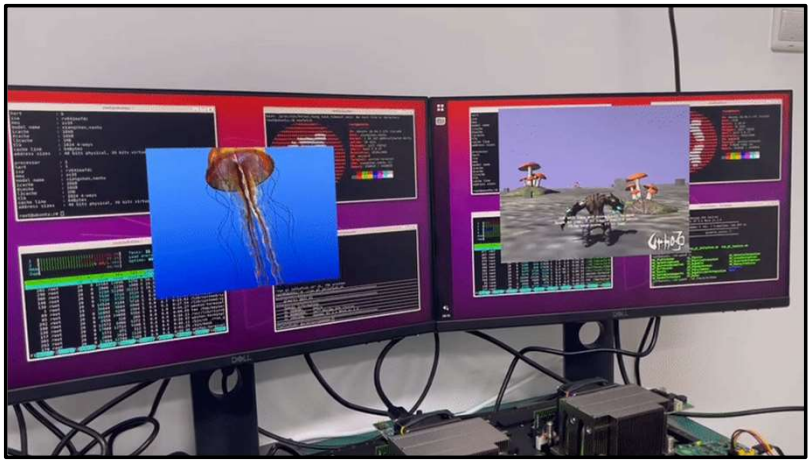


Feature	V1 YQH	V2 NH
ISA	RV64GC	RV64GCBK
Process Node	28nm	14nm
Core Count	1	2
Die Size	8.6 mm^2	22.13 mm^2
Std Cell Num/Area	5.05M, 4.27 mm^2	11.3M, 4.53 mm^2
Mem Num/Area	261, 1.7 mm^2	692, 8.93 mm^2
Density	66%	35%
Frequency	1.3GHz, TT 0.9V	2GHz, TT 0.9V

* Updated January 5, 2023. Performance confirmed on FPGA as well

FPGA Prototype in Only Two Weeks



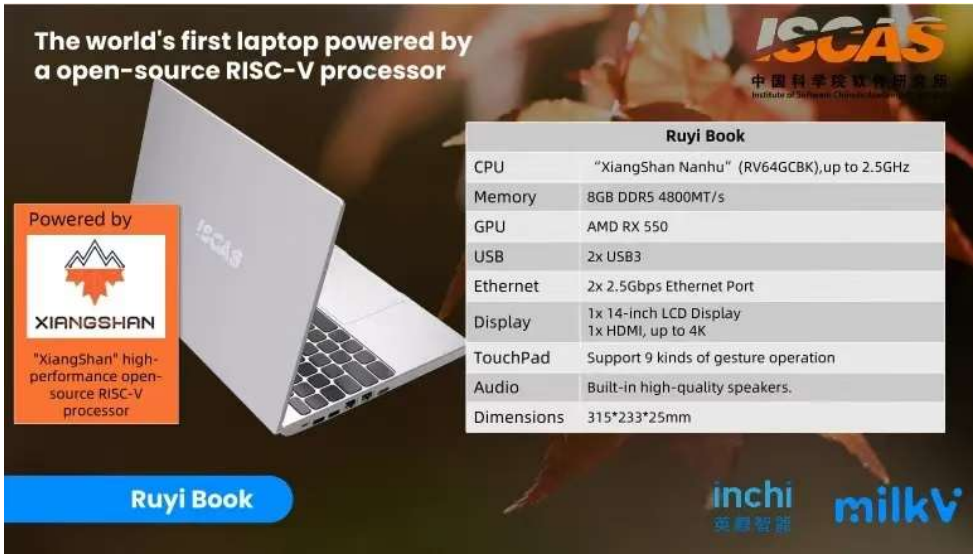


Source: Xinchen Technology

Real Chip of XiangShan Gen 2 Nanhu

- **Test chip was back in October 2023**
  - Successfully brought up Linux and working with PCIe device (GPU, Ethernet, USB..)

- Nanhu test board @ RVSC'24
  - Running Desktop Linux & Cloud Game*



- Ruyi XiangShan Book
  - Design by ISCAS, Inchi, MilkV

* Genshin Impact · Cloud, only ~ 1 fps, just for fun

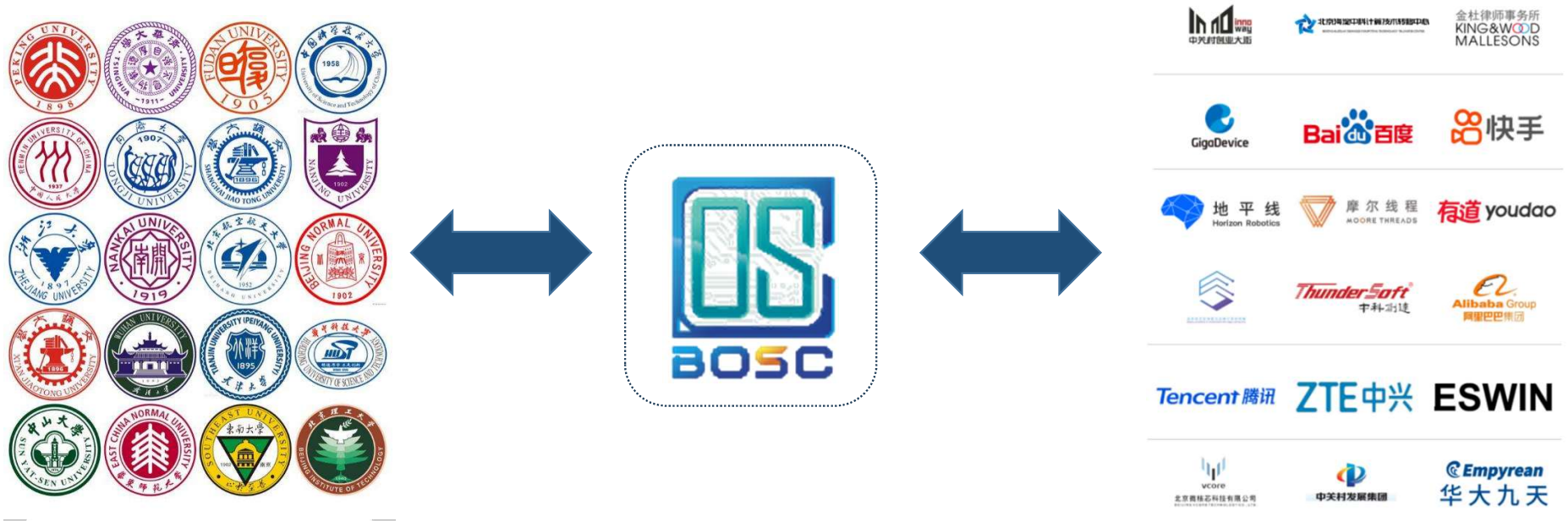
XiangShan Gen 3: Kunminghu

- **Target ARM Neoverse N2**
  - SPECCPU2006: 45@3GHz (15/GHz)
  - Vector/Hypervisor extension supported
- **A Joint Development Team**
  - coordinated by Beijing Institute of Open-Source Chip

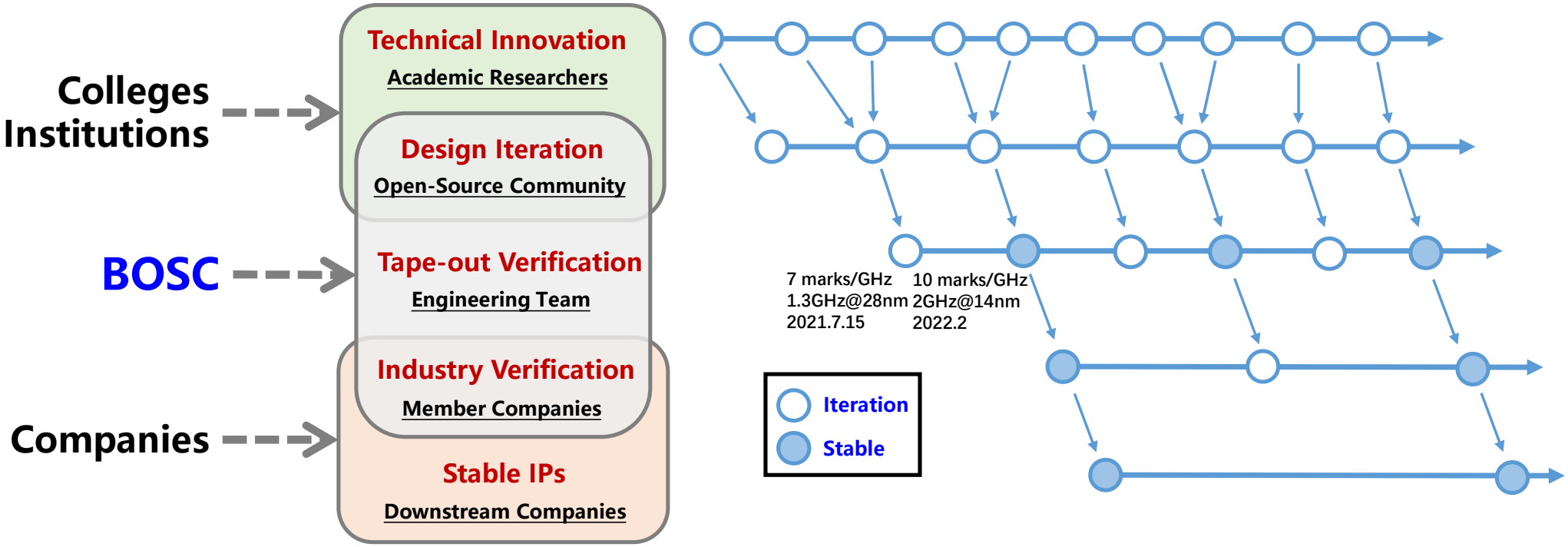
Kunming Lake in Beijing

Collaborative Mechanism Based on Open-Source Innovation

- With high-level open-source hardware,
  we gather global innovative forces from various colleges and institutions,
  engaging in collaborative verification with the industry.

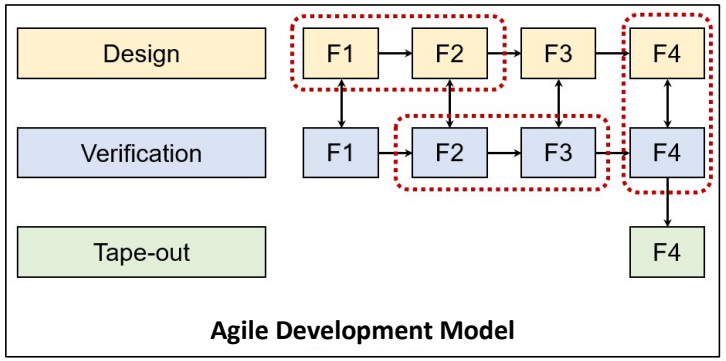


Tiered Rolling Open-Source Model



Highlights in XiangShan Gen 3 Kunminghu

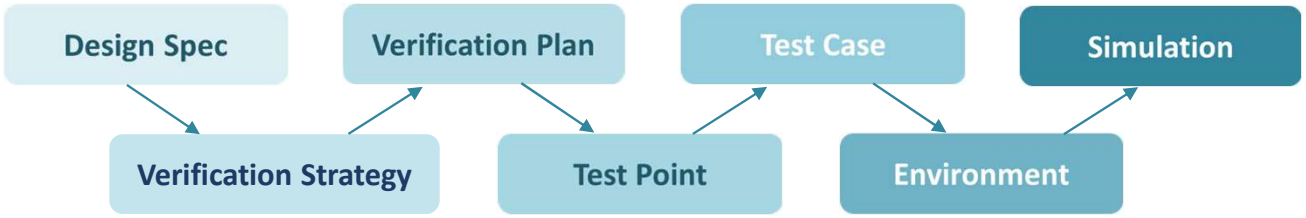
- **Functional Enhancement**
  - Support RISC-V **Vector/Hypervisor** extension
  - Support **RVA23 profile**
  - Support interconnection based on **CHI** protocol



- **Performance Exploration**
  - Performance boost in frontend, backend, load-store unit and cache
  - Performance model **XS-Gem5** calibrated with RTL
  - Workflow: **DSE on perf model => Impl. & fine tunning on RTL**
- **Physical Design**
  - Experienced physical design team
  - Simultaneous iteration of RTL coding based on timing evaluation
- **Functional Verification**
  - **Hierarchical verification flow** including system/integration/unit level + FPGA prototyping
  - Industrial-grade verification process

Industrial-grade Verification Process

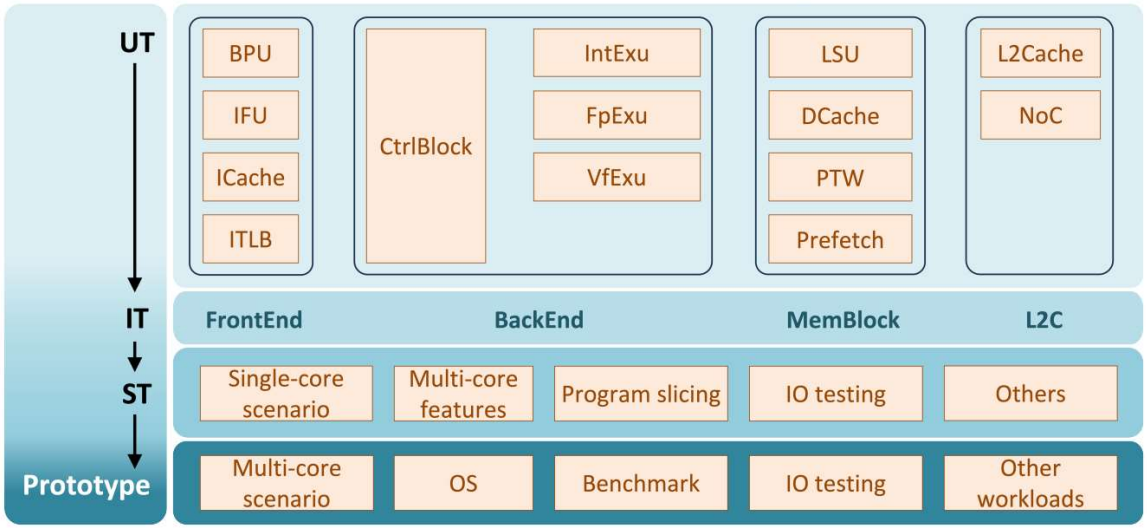
① Process



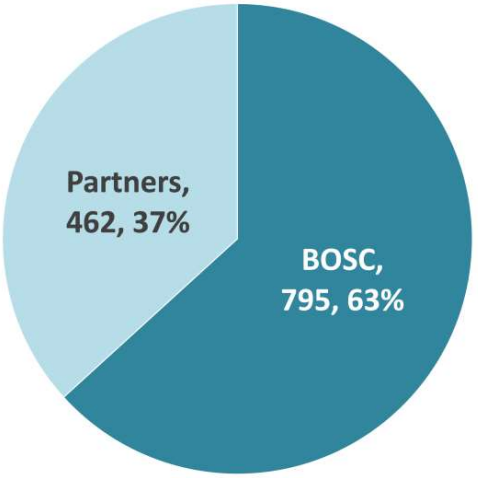
② Tools

UVM UnityChip XFuzz
SPIKE DiffTest FORCE-RISCV
RISCV-DV EMU FPGA SIM
EDA Business tools Formal

③ Verification Hierarchy



④ Partner Collaboration

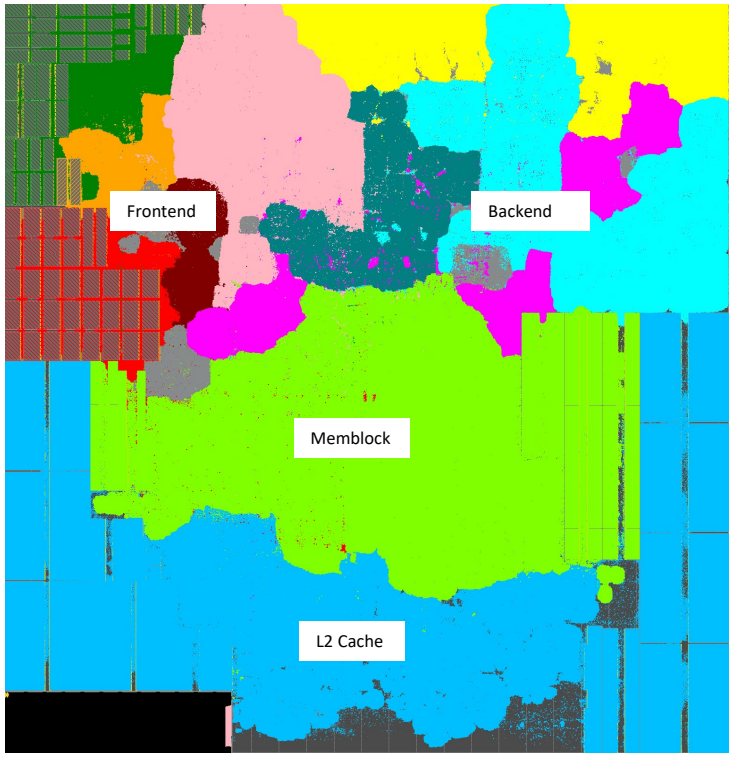


Source and Number of Reported Bugs

Performance Evaluation of Gen 3 Kunminghu

- Method: SPEC CPU checkpoints selected by Simpoint
  - 1MB L2 and 16MB L3

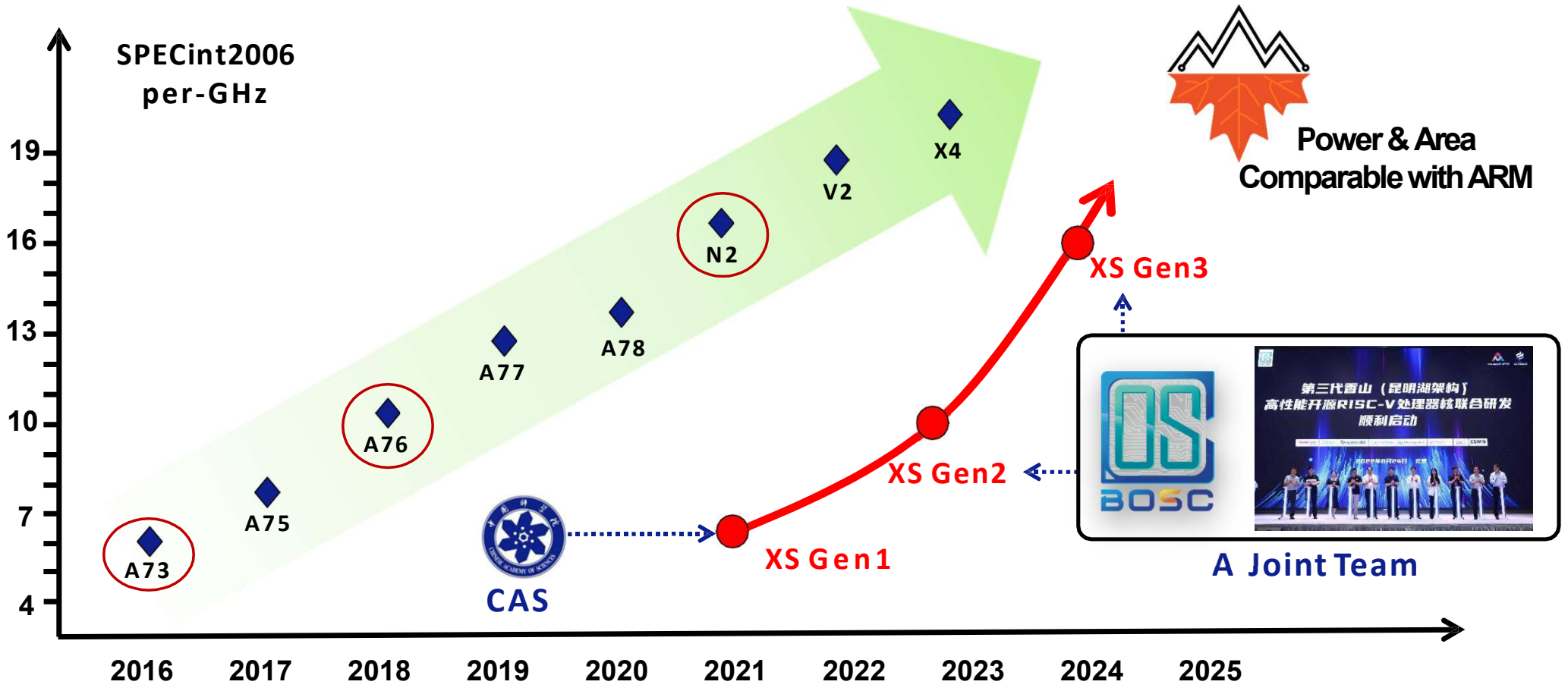
SPECint 2006 est. @ 3GHz		SPECfp 2006 est. @ 3GHz	
400.perlbench	35.91	410.bwaves	66.86
401.bzip2	25.51	416.gamess	40.89
403.gcc	47.87	433.milc	45.17
429.mcf	59.56	434.zeusmp	52.09
445.gobmk	30.31	435.gromacs	33.66
456.hmmer	41.61	436.cactusADM	46.18
458.sjeng	30.52	437.leslie3d	45.90
462.libquantum	122.43	444.namd	28.88
464.h264ref	56.58	447.dealII	73.57
471.omnetpp	41.42	450.soplex	52.63
473.astar	29.30	453.povray	53.39
483.xalancbmk	72.95	454.Calculix	16.38
GEOMEAN	**44.58**	459.GemsFDTD	35.68
SimPoint Based Sampling GCC 12 -O3, RV64GCB, jemalloc DRAMsim3 DDR4 @ 3200MHz		465.tonto	36.68
		470.lbm	91.19
		481.wrf	40.64
		482.sphinx3	49.11
		GEOMEAN	**44.54**



Floorplan of KMH V2R2 (single core)

* Updated in May 2025

XiangShan: Open-Source High Performance Processors



Prospect: "Dual Core" Roadmap

Based on **V3 (KMH)**

- **Big Core: Ultimate Performance （v.s. ARM N2)**

Target High-Throughput Advanced Computing Platform

Goal： become mainstream CPU for data centers and computational facilities

Based on **V2 (NH)**

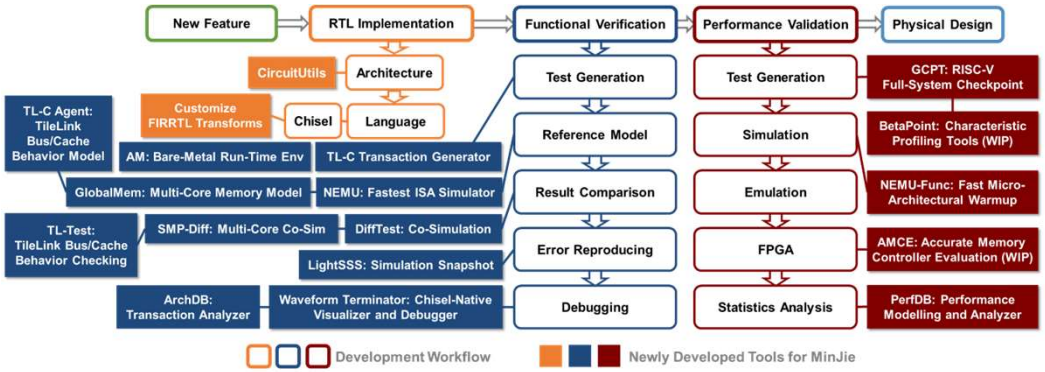
- **Mid Core: Balanced Perf & Efficiency (v.s. ARM A76)**

Target Mid-to-High-End General Industry Domain

Goal: support broad industrial spectrum including industrial control, automotive, communication, aviation and more

Part III

MinJie: Agile Development for High-Performance RISC-V Processors



First Step to Agile Design: Use Chisel

- **2018: quantitative experiments between Chisel and Verilog**

- **Task #1:** *Design an L2 Cache for RISC-V Rocket-chip core*
- **Who:** *A 5-year engineer vs. a senior student*

	A 5-year Engineer	An Undergraduate
Experience	Familiar w/ OpenSparc T1; Modified Xilinx Cache	Course projects on CPU design; Using Chisel for 9 months
Language	Verilog	Chisel
Time	6 weeks	3 days
LOCs	~1700	~350
Results	Unable to boot Linux	Boot Linux on **multicore RISC-V**; support **NIC w/ DMA** mode

- **1st Round results:** *Chisel is more productive than Verilog by **14X** with only **1/5** LOC*

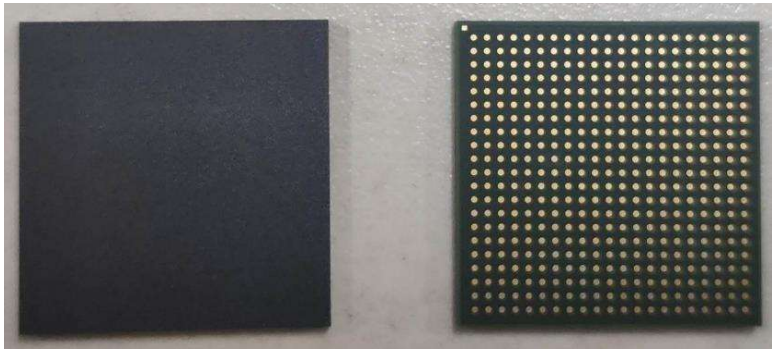
- **Task #2**: *Translate the Verilog codes into Chisel*
  - Evaluated on FPGA (xc7v2000tfhg1716-1), Vivado 2017.01
- **Who**: *A junior student who never knew Chisel*

	Verilog	Chisel (direct translation)	Chisel-opt (adv. features & libs)
Freq./MHz	135.814	136.388 (+0.42%)	154.107 (+13.47%)
Power/W	0.770	0.749 (-2.73%)	0.749 (-2.73%)
LUT Logic	5676	6422 (+13.14%)	2594 (-54.30%)
LUT Storage	1796	1264 (-29.62%)	1492 (-16.93%)
FF	4266	3638 (-14.72%)	747 (-82.49%)
LOCS	618	470 (-23.95%)	155 (-74.92%)

- **2nd Round results:** *Chisel can achieve better PPA than verilog*

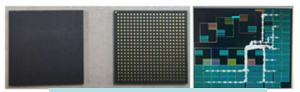
Yu Zihao, Liu Zhigang, Li Yiwei, Huang Bowen, Wang Sa, Sun Ninghui, Bao Yungang. Practice of Chip Agile Development: Labeled RISC-V. Journal of Computer Research and Development, 2019, 56(1): 35-48.

- **2020: 28-nm tape-out of an 8-core labeled RISC-V processor**



16-node prototype case

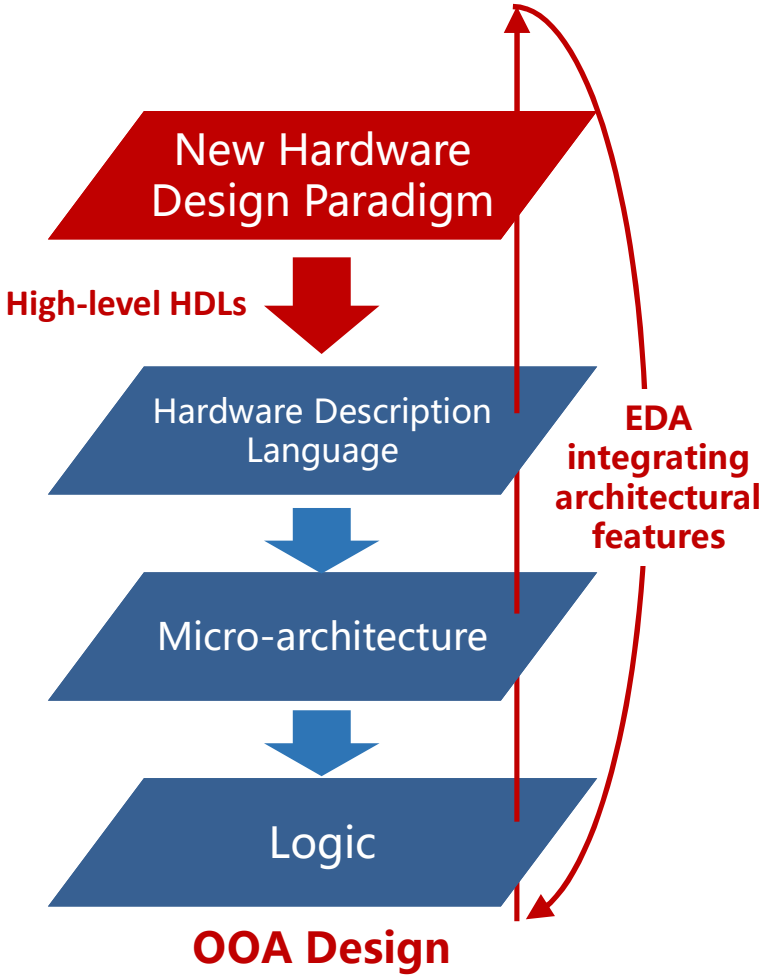
single node board



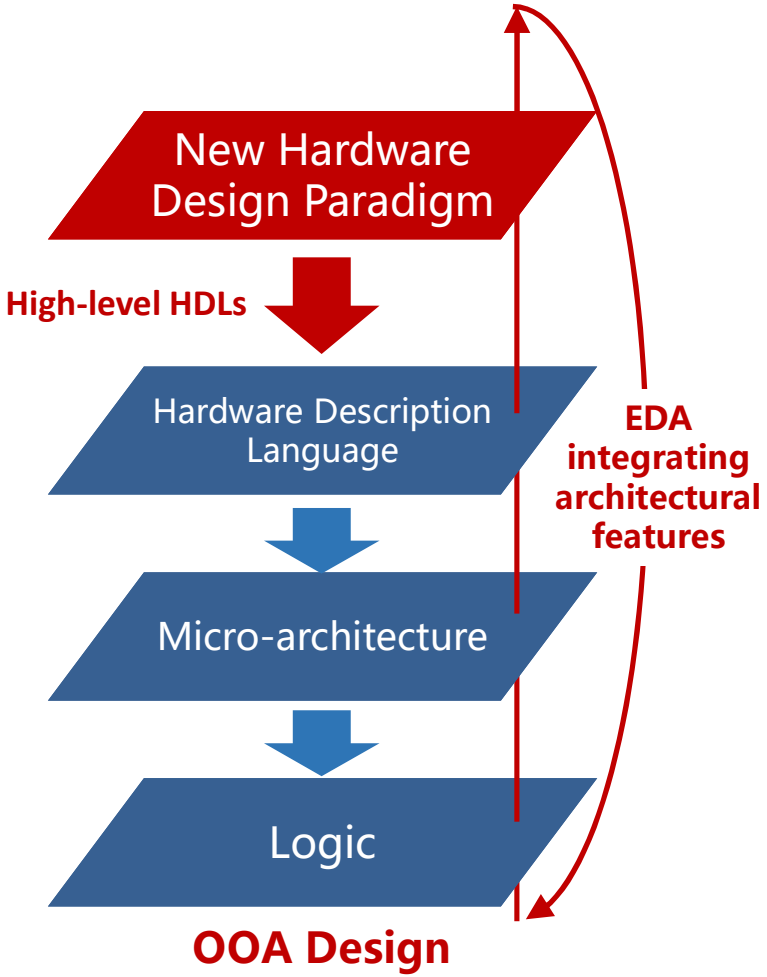
"BeiHai 100" Labeled RISC-V Chip

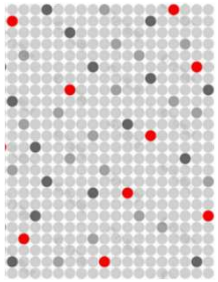
- RV64GC指令集
- 单发射顺序9级流水线
- 内置标签化冯诺依曼结构技术
- 8核/2MB L2 Cache
- ChipLink前端总线
- **1.2GHz@ 28nm**
- Wafer out/WB BGA封装
- 最大支持32GB DDR4内存
- 2*千兆以太网
- 1*PCIe3.0 RC x4

New HDL → New Design Paradigm



New HDL → New Design Paradigm

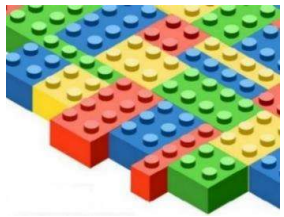




Pixel
(gate-level)

Circuit-oriented Processor Design

Processor



Object
(Reusable Modules)

Object-oriented Processor Design

Processor

Configurable

What's Missing in Agile Hardware Design? Verification!

What's Missing in Agile Hardware Design? Verification!

Babak Falsafi, *Fellow, ACM, IEEE*

Parallel Systems Architecture Laboratory, Institute of Computer and Communication Sciences, School of Computer and Communication Sciences, Ecole polytechnique fédérale de Lausanne CH-1015 Lausanne

E-mail: babak.falsafi@epfl.ch

Agile hardware design is an approach to developing hardware systems that draws inspiration from the principles and practices of agile software development. It emphasizes collaboration, flexibility, iterative development, and quick adaptation to changing requirements. In agile hardware design, the focus is on delivering functional hardware systems in shorter development cycles while maintaining high-quality and customer satisfaction.

In particular, agile hardware design is of great interest in the open-source hardware community. Open-source hardware development —such as RISC-V— is at the forefront of initiatives to democratize hardware and drive innovation in chip design forward. Agile design is instrumental for the RISC-V community because it supports rapid iteration, accommodates the evolving RISC-V standard and the addition of custom extensions, improves community collaboration and time-to-market, and addresses the design challenges associated with complex architectural features.

Among significant innovations based on agile hardware design is the recently announced XIANGSHAN RISC-V core which is currently the highest performing RISC-V out-of-order microprocessor core with single-thread performance exceeding both existing RISC-V cores and a state-of-the-art ARM core, Cortex-A76. The creators of this platform have published their agile design methodology in a flagship computer architecture venue, MICRO, with a paper that has been selected through peer review to be among the best dozen papers in all of computer architecture in one year for publication in IEEE Micro Top Picks.

A key contributor to this breakthrough has been integrating hardware verification into the agile methodology. Hardware verification is crucial in designing digital platforms, as it ensures that semiconductor chips operate correctly and reliably according to the architecture specifications. Verification guarantees compliance with standards, and helps detect and rectify design errors, validate system-level functionality, optimize performance and power consumption, and enhance hardware reliability and safety. It plays a fundamental role in creating robust and dependable CPUs that meet the requirements of various applications and workloads.

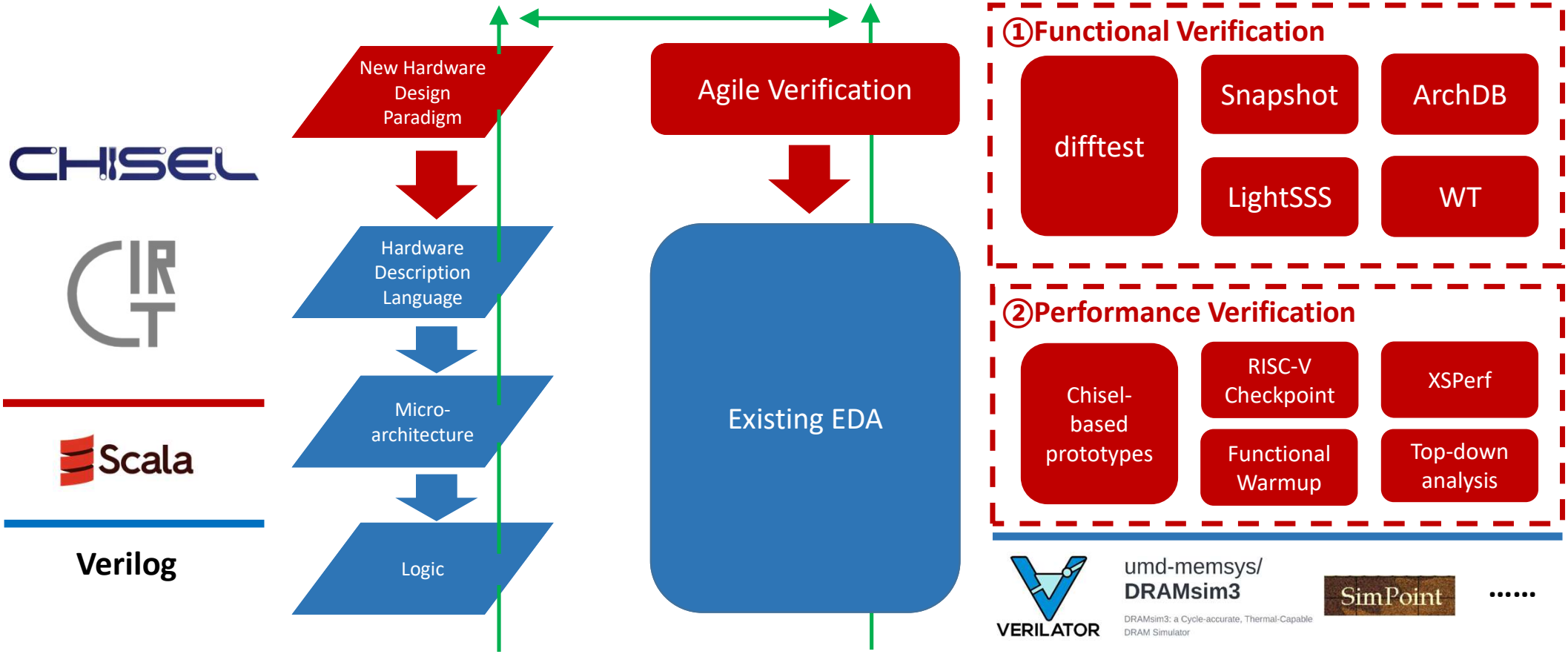
Babak Falsafi

Professor, EPFL

ACM/IEEE Fellow

* Babak Falsafi. What's missing in agile hardware design? Verification! JOURNAL OF COMPUTER SCIENCE AND TECHNOLOGY 38(4): 735−736 Jul. 2023.

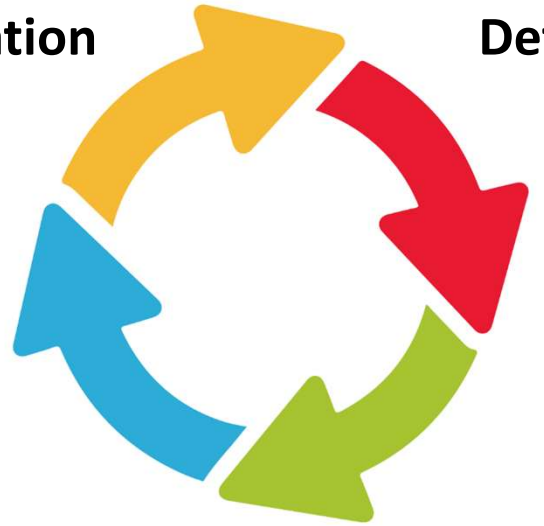
Agile Verification is Challenging



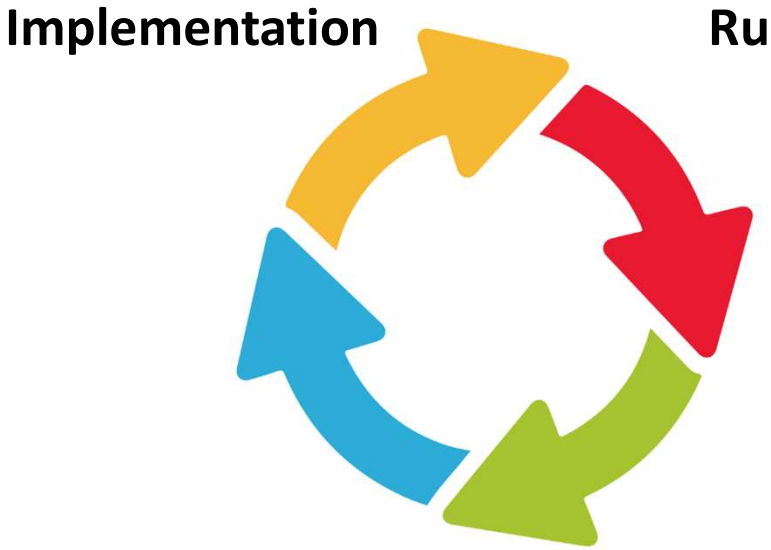
Agile Design Languages | Agile Design Method | Agile Verification Method | Agile Verification Tools

MinJie Loop

Functional Verification Loop

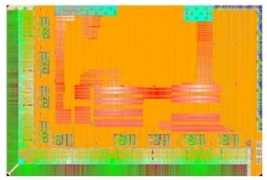


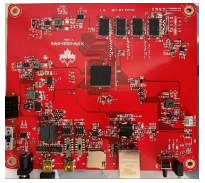
Performance Optimization Loop

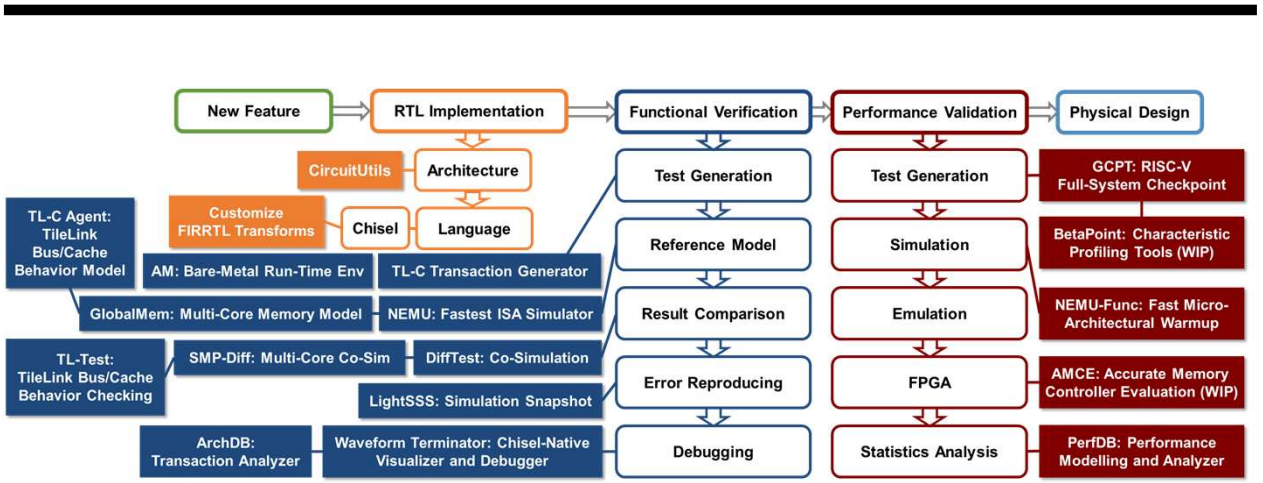


Minjie: Open & Agile Verification Toolchain

- **Infrastructure** is the key outcome of the XiangShan Project
- **Open source** to benefit both academia and industry

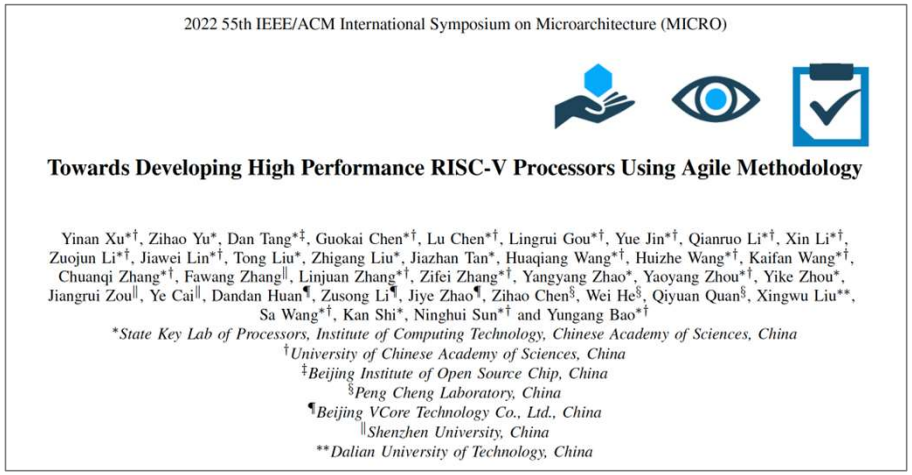






Advanced Agile Chip Design Methodology

- New method and toolchain for agile chip design
- Developed more than 17 new tools to solve agile verification problems
- **MICRO 2022 → IEEE Micro Top Picks**



2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)

Towards Developing High Performance RISC-V Processors Using Agile Methodology

Yinan Xu*†, Zihao Yu*, Dan Tang*‡, Guokai Chen*†, Lu Chen*†, Lingrui Gou*†, Yue Jin*†, Qianruo Li*†, Xin Li*†, Zuojun Li*†, Jiawei Lin*†, Tong Liu*, Zhigang Liu*, Jiazhan Tan*, Huaqiang Wang*†, Huizhe Wang*†, Kaifan Wang*†, Chuanqi Zhang*†, Fawang Zhang‖, Linjuan Zhang*†, Zifei Zhang*†, Yangyang Zhao*, Yaoyang Zhou*†, Yike Zhou*, Jiangrui Zou‖, Ye Cai‖, Dandan Huan¶, Zusong Li¶, Jiye Zhao¶, Zihao Chen§, Wei He§, Qiyuan Quan§, Xingwu Liu**, Sa Wang*†, Kan Shi*, Ninghui Sun*† and Yungang Bao*†

*State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, China
†University of Chinese Academy of Sciences, China
‡Beijing Institute of Open Source Chip, China
§Peng Cheng Laboratory, China
¶Beijing VCore Technology Co., Ltd., China
‖Shenzhen University, China
**Dalian University of Technology, China

THEME ARTICLE: TOP PICKS FROM THE 2022 COMPUTER ARCHITECTURE CONFERENCES

Toward Developing High-Performance RISC-V Processors Using Agile Methodology

Yinan Xu and Zihao Yu, *State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, 100190, China*

Dan Tang, *Beijing Institute of Open Source Chip, Beijing, 100080, China*

Ye Cai, *Shenzhen University, Shenzhen, 518060, China*

Dandan Huan, *Beijing VCore Technology, Beijing, 100190, China*

Wei He, *Peng Cheng Laboratory, Shenzhen, 518060, China*

Ninghui Sun and Yungang Bao, *State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, 100190, China*

XiangShan achieves L2.5

L1: OPEN ISA ✓

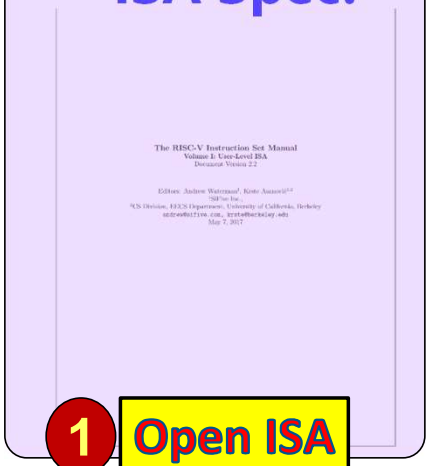
L2: OPEN Design/Implementation ✓

L3: OPEN Framework/Tools ✓

3 **Open Framework/Tools**

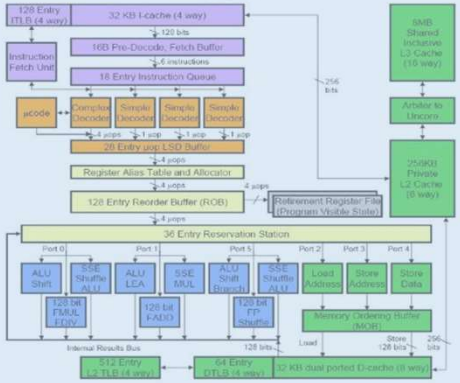
microarch. → coding → EDA Tools

ISA Spec.



1 **Open ISA**

Docs



RTL

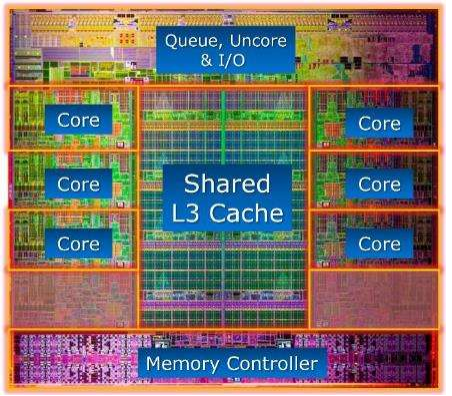
```
component DebugCoreTop is
  port (
    -- Trigger and Data
    cu_Clk      : in    std_logic_vector(2 downto 0) := (others => '0');
    cu0_Trig    : in    t_trig_0 := (others => (others => '0'));
    cu1_Trig    : in    t_trig_1 := (others => (others => '0'));
    cu2_Trig    : in    t_trig_2 := (others => (others => '0'));
    cu0_Data    : in    t_data_0 := (others => (others => '0'));
    cu1_Data    : in    t_data_1 := (others => (others => '0'));
    cu2_Data    : in    t_data_2 := (others => (others => '0'));

    -- Downstream I2C
    SCL         : in    std_logic := '0';
    SDA         : inout std_logic := '0';

    -- Upstream
    gt_RefClk_p : in    std_logic := '0';
    gt_RefClk_n : in    std_logic := '0';
    gt_RX_p     : in    std_logic_vector(2 downto 0) := (others => '0');
    gt_RX_n     : in    std_logic_vector(2 downto 0) := (others => '0');
    gt_TX_p     : out   std_logic_vector(2 downto 0);
    gt_TX_n     : out   std_logic_vector(2 downto 0)
  );
end component;
```

2 **Open Design/Implt**

Layout

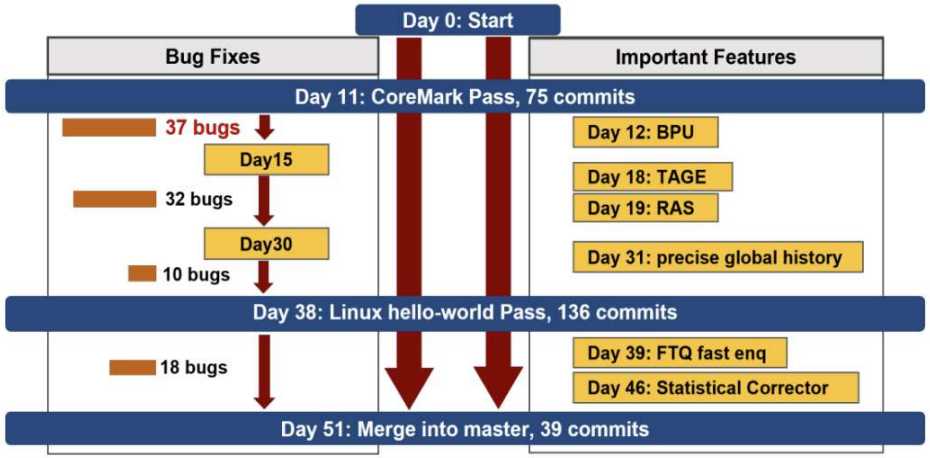


Part IV

An Effective Infrastructure for Research

An Effective Infrastructure for Research

① Micro-architecture Optimization



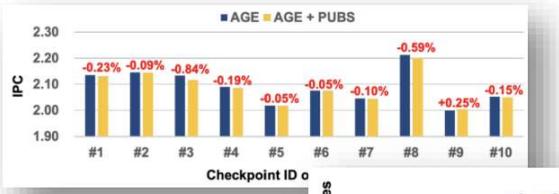
- 3 graduate students
- 11 days for a functionally correct prototype
- 37 bugs in 5 days
- 38 days to boot Linux with BPU
- 51 days for the overall frontend architecture

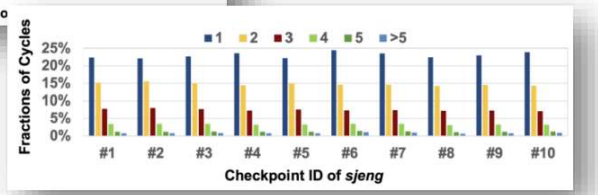
② Paper Reproduction

2018 51st Annual IEEE/ACM International Symposium on Microarchitecture

Performance Improvement by Prioritizing the Issue of the Instructions in Unconfident Branch Slices

One third-year PhD student, 200 minutes on XiangShan





An Effective Infrastructure for Research

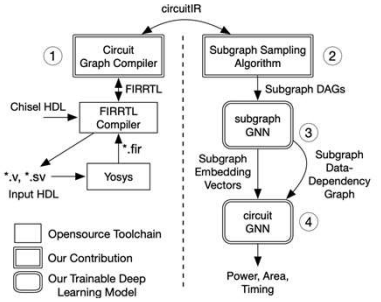
- ***Topic: Computer Architecture***
  - XiangShan: a realistic out-of-order RISC-V implementation with industry-competitive performance and an active open-source community
  - MinJie provides the toolchains
- *Microarchitecture, accelerators, novel architectures, profiling, systems, benchmarking, security, compilers, …*

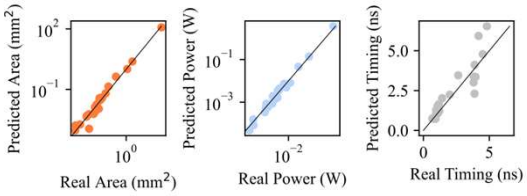
midgard EPFL

https://midgard.epfl.ch/

Imprecise Store Exceptions, ISCA'23 (EPFL)

- ***Topic: Agile Chip Development***
  - XiangShan is a progressive, configurable, complicated, challenging benchmark
  - MinJie provides a good startpoint
- *HDLs, verification, performance, power, area, prototyping, DFT, synthesis, placement, routing, ECO, …*





SNS v2, MICRO'23 (Duke University)

北京开源芯片研究院 BEIJING INSTITUTE OF OPEN SOURCE CHIP

中国开放指令生态（RISC-V）联盟 China RISC-V Alliance

香山开源处理器社区 XiangShan Open-Source Processor Community

Summary

- **Era of open-source chip is coming**
  XiangShan fills the gap in high performance open source processors.

- **Three generations, dual-core roadmap**
  XiangShan meet the needs from both academia and industry.

- **An effective platform for research on**
  microarchitecture and agile hardware design.

Thanks!

What we will cover in this tutorial

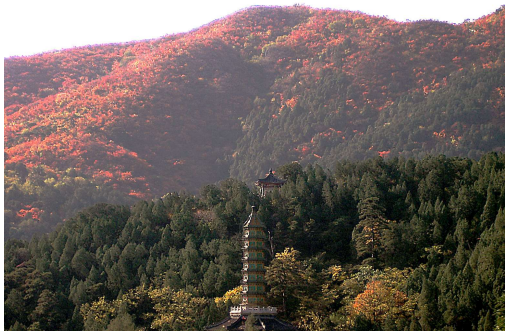
- Introduce of XiangShan (11:30 – 12:00, 30 minutes)

- **CPU Microarchitecture (12:00 – 12:30, 30 minutes)**
  - *Design and implementation – How to implement novel ideas on XiangShan*
  - Frontend: branch prediction and instruction fetch
  - Backend: out-of-order scheduler, execution units
  - Load/Store Unit: LSQ, pipelines, TLBs, data caches
  - L2/L3 caches and prefetchers

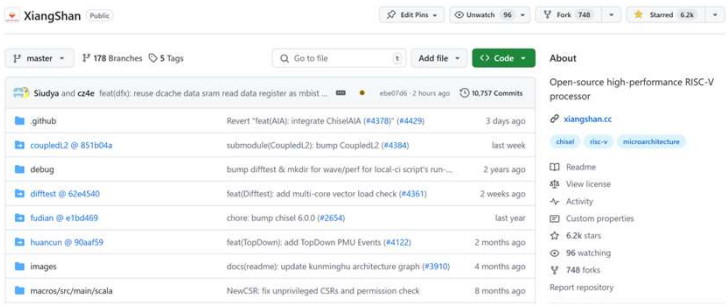
- Development workflows (12:30 – 13:00, 30 minutes)
  - *Introduction of their usages – How to develop on XiangShan with MinJie*
  - Simulation and Debugging
  - Research Demo

XiangShan: Open-Source High-Performance Processor

- **1st generation: YQH (Yanqihu)**
  - 2020/6: first commit of design RTL
  - 2021/7: **28nm tape-out, 1.3GHz**
  - Performance: **SPEC CPU2006 7.01@1GHz, DDR4-1600**
- **2nd generation: NH (Nanhu)**
  - 2021/5: starting design exploration and RTL design
  - 2023/4: GDSII delivery
  - **14-nm tape-out, estimated SPEC CPU2006 20@2GHz**
- **3rd generation: KMH (Kunminghu)**
  - ISA feature: **Vector (V) extension, Hypervisor (H) extension**
  - Support RVA23 profile and L2 Cache with CHI protocol
  - **estimated SPEC CPU2006 45@3GHz**
- **Open-sourced at https://github.com/OpenXiangShan/XiangShan**



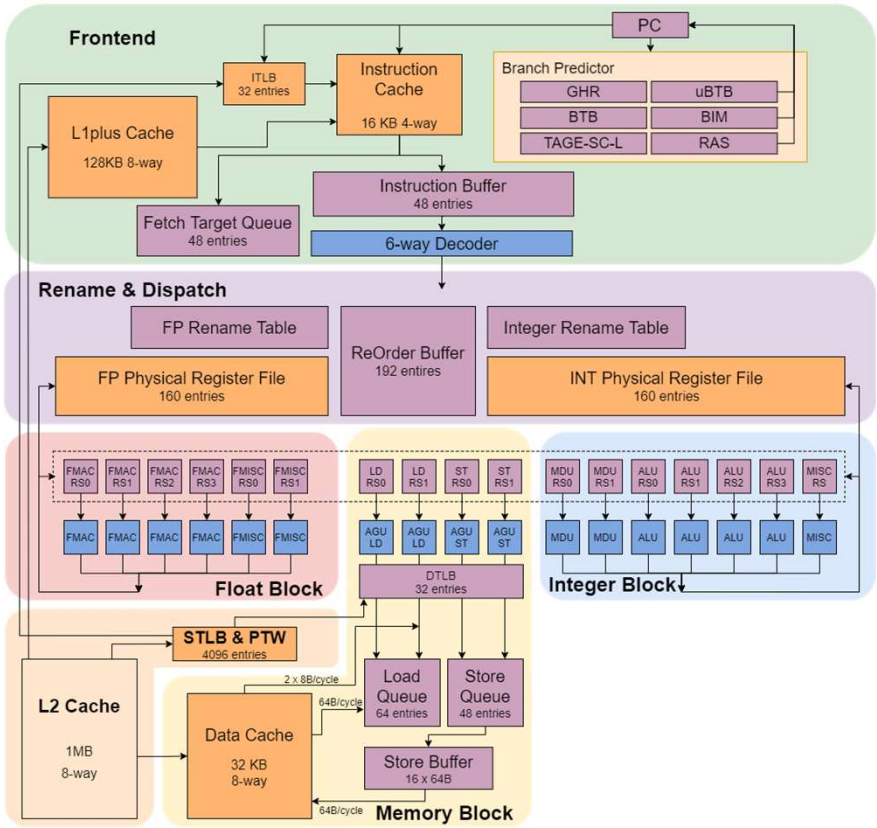
Fragrant Hills in Beijing



> 6.2K stars, > 740 forks on GitHub

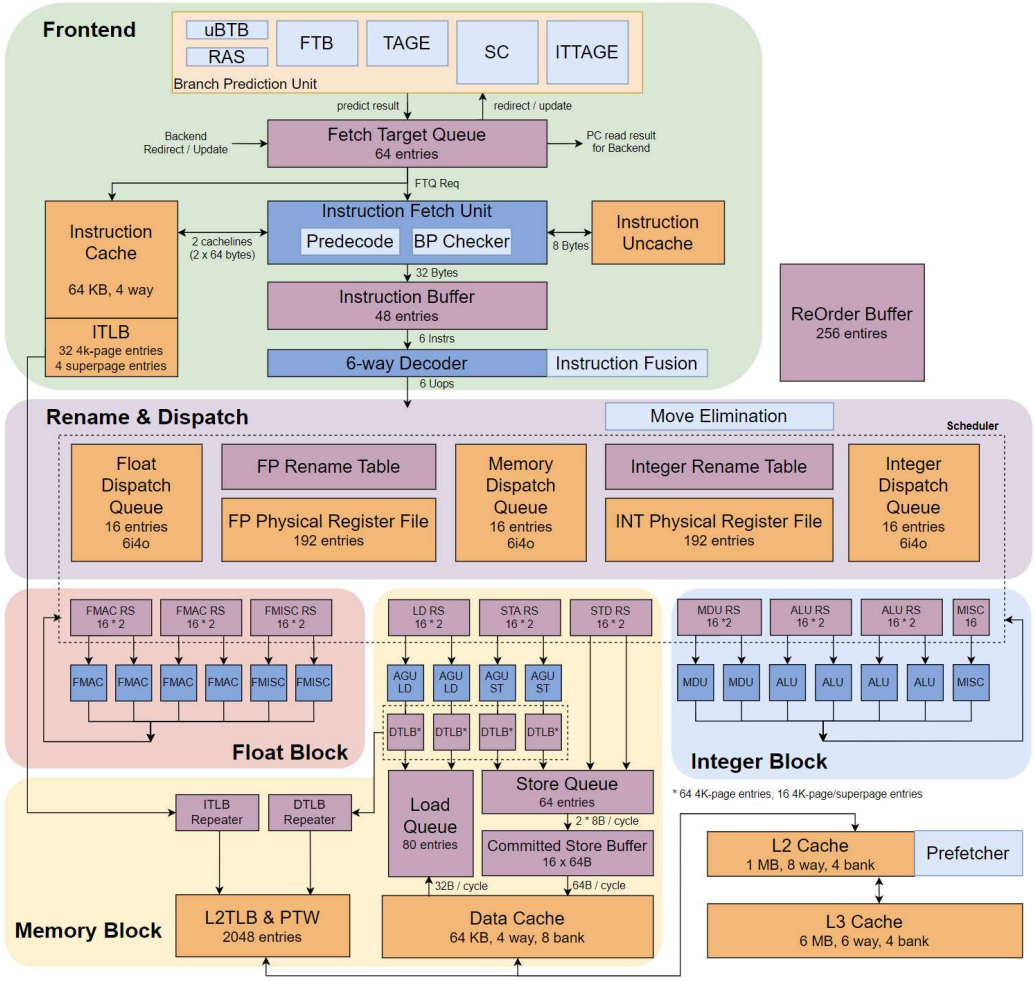
Yanqihu : 1st generation of XiangShan

- **11**-stage, **6**-wide decode/rename
- **TAGE-SC-L** branch prediction
- **160** Int PRF + **160** FP PRF
- **192**-entry ROB, **64**-entry LQ, **48**-entry SQ
- **16**-entry RS for each FU
- **16KB** L1 Cache, **128KB** L1plus Cache for instruction
- **32KB** L1 Data Cache
- **32**-entry ITLB/DTLB, **4K**-entry STLB
- **1MB** inclusive L2 Cache



Nanhu: 2nd generation of XiangShan

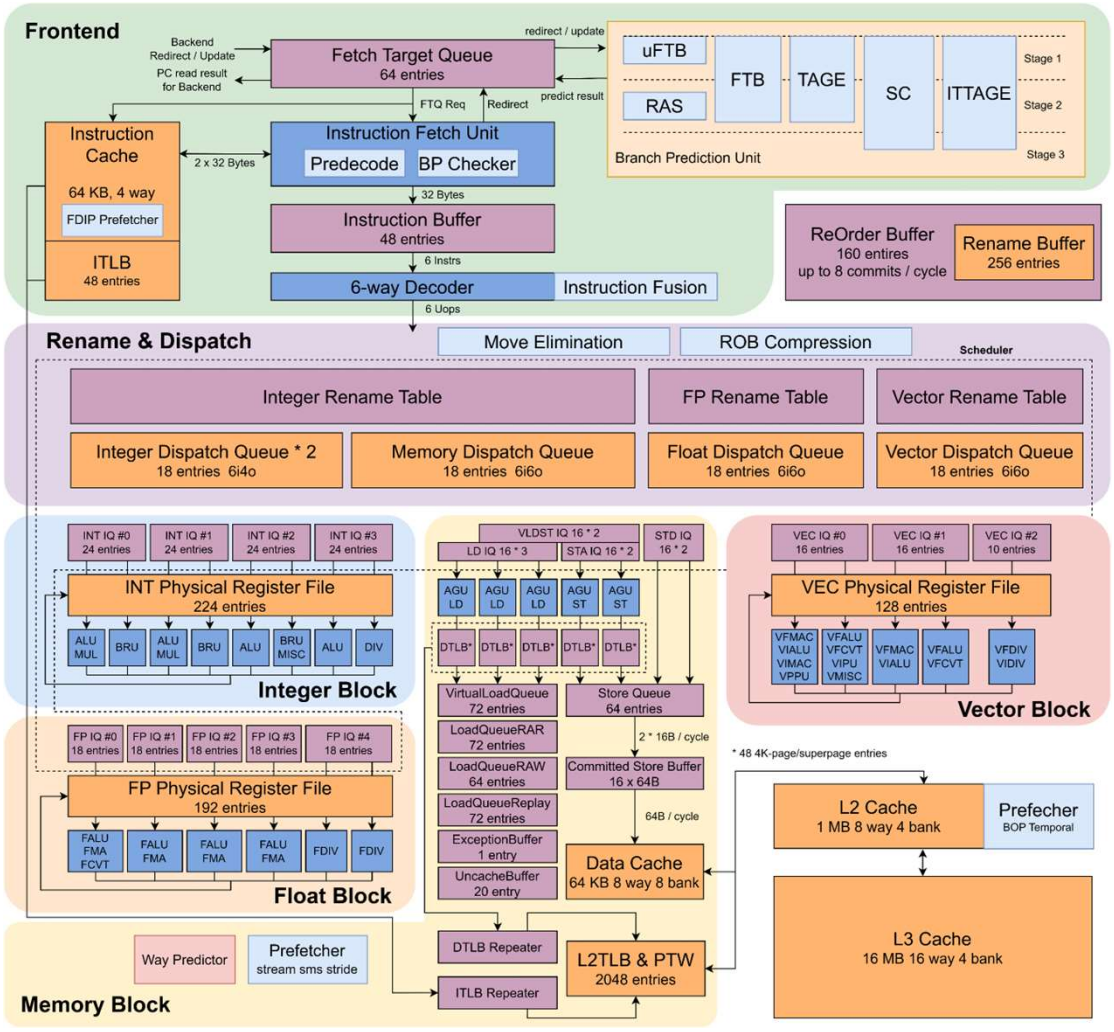
- **192** Int PRF + **192** FP PRF
- **256**-entry ROB, **80**-entry LQ, **64**-entry SQ
- **32**-entry RS for two FUs
- **64KB** L1 Instr. Cache, **64KB** L1 Data Cache
- **64**-entry DM + **16**-entry FA DTLB
- **32**-entry ITLB, **2K**-entry STLB
- **1MB non-inclusive** L2 Cache
- **6MB** non-inclusive L3 Cache (LLC)



Overview: XiangShan Gen 3 Kunminghu (KMH)

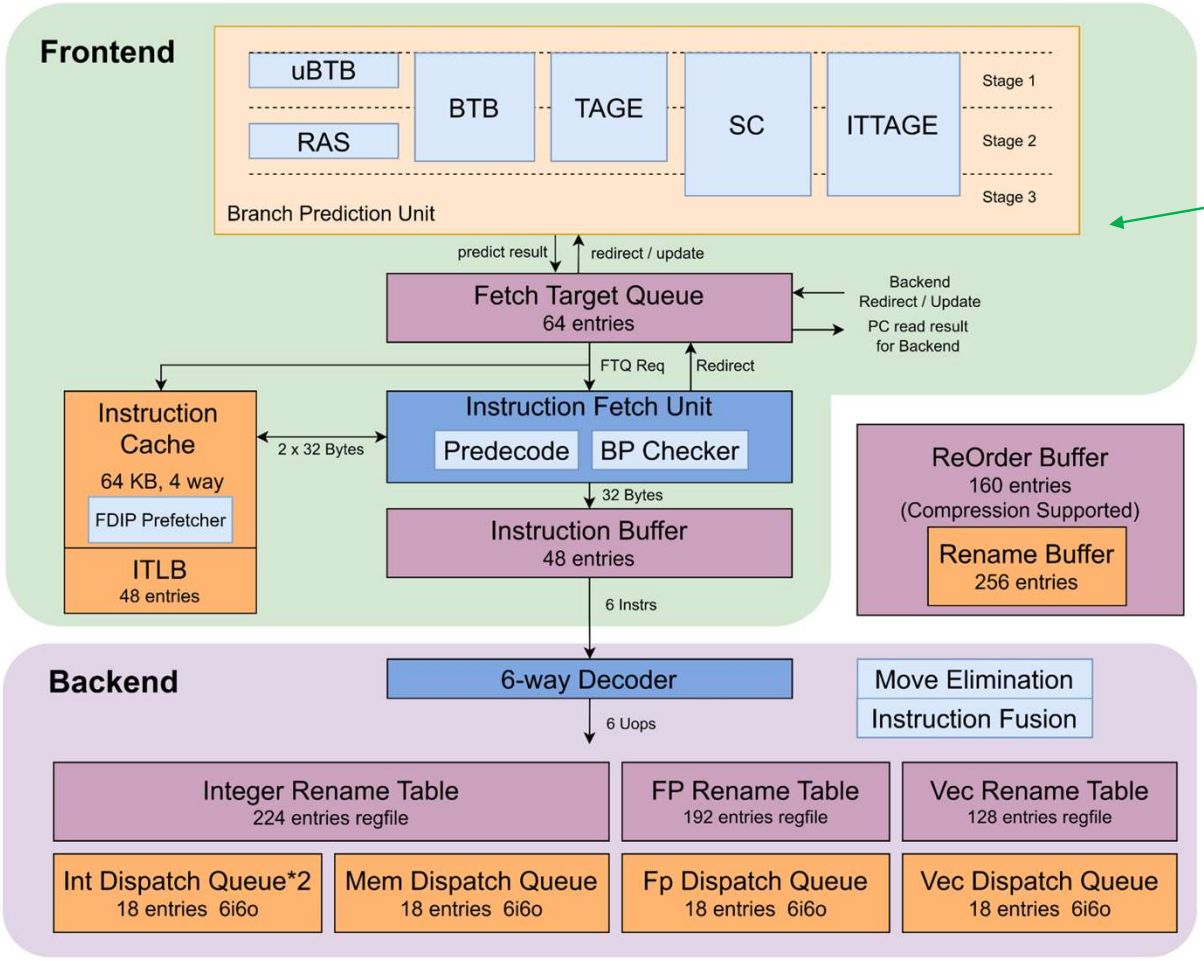
- **Work with Beijing Institute of Open Source Chip (BOSC)**
  - Joint validation with partner companies based on open source
- **Functional Improvement**
  - Support RISC-V **RVA23 profile**
  - RISC-V **Vector** and **Hypervisor** extension implementation
- **Performance Exploration**
  - Performance upgrade of Frontend, Backend, load-store unit and cache
  - DSE on performance model (**XS-Gem5**) calibrated with RTL
- **Functional Verification & Physical Design**
  - Hierarchical verification flow including **UT, IT, ST + FPGA**
  - Industrial-grade verification process
  - Simultaneous iteration of RTL coding with physical design timing evaluation

3rd Kunminghu µArch Overview



- **Decoupled frontend**
  - Reduce fetch bubbles
  - Instruction prefetching friendly
- **Aggressive outstanding instruction window**
  - Large ROB/LQ/SQ
- **Low latency & high BW $ access**
  - Closely-coupled load/store pipe & L1/L2$
  - Highly-banked design
  - Multi-level composite prefetching
- **ISA support**
  - Support RISC-V RVA23 profile
  - RVWMO (RISC-V Weak Memory Ordering)

3rd Kunminghu µArch Overview

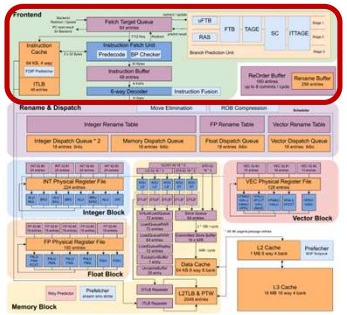


Multi-level branch predictors

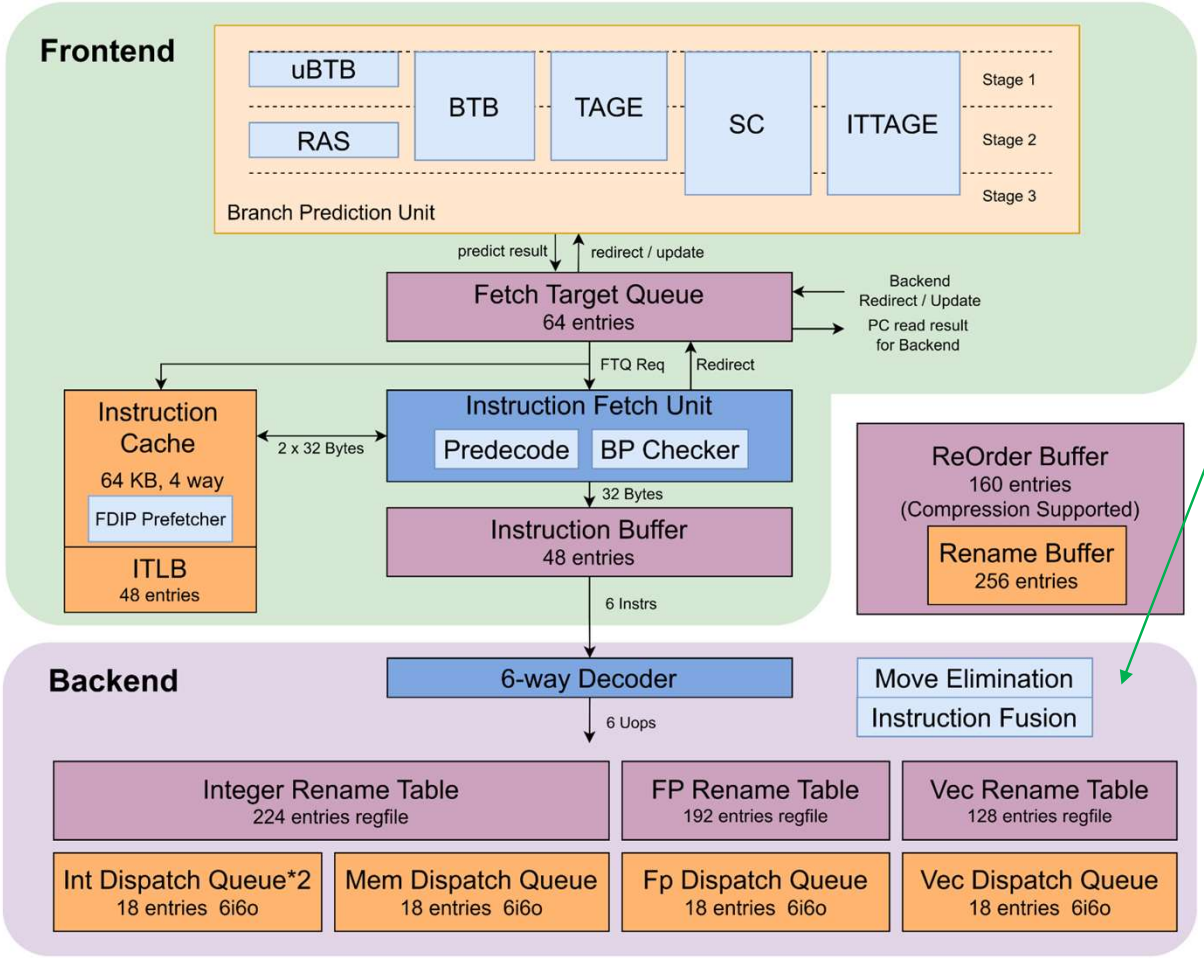
- **256**-entry µBTB
- **2K**-entry BTB, optional L2 BTB
- **16K** TAGE-SC direction predictor
- **2K** ITTAGE indirection predictor
- **48**-entry return address stack

Instruction cache and TLB

- **64KB 4-way** ICache
- **48**-entry ITLB
- Fetch-directed instruction prefetcher



3rd Kunminghu µArch Overview



6-wide decode/rename/dispatch

Register rename

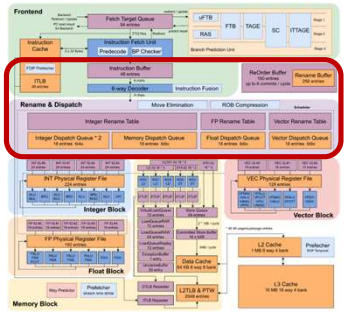
- **224**-entry Int regfile
- **192**-entry FP regfile
- **128**-entry Vec regfile
- Move elimination
- Instruction fusion

160 entry ROB

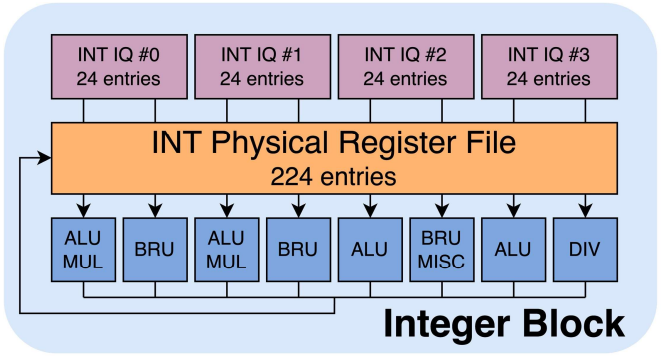
- Support compression (up to **6**-µop/entry)
- **8**-entry retire/cycle
- Recovery via checkpoint + rollback

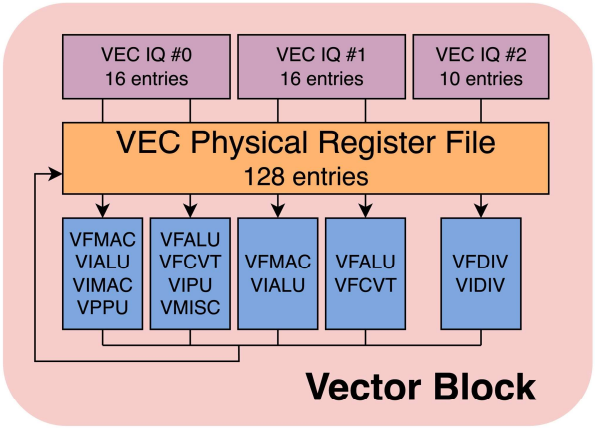
256 entry Rename Buffer

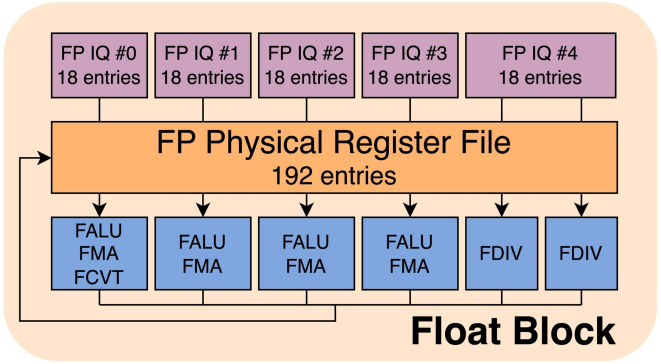
- Bridge the gap between commit & rename table update



3rd Kunminghu µArch Overview







Integer block

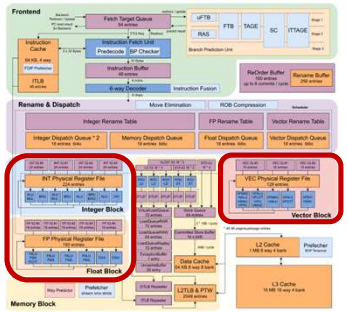
- **4** ALU (2 with 3-stage multiplier)
- **3** BRU for branch resolution
- **1** SRT radix-16 divider

Float-point block

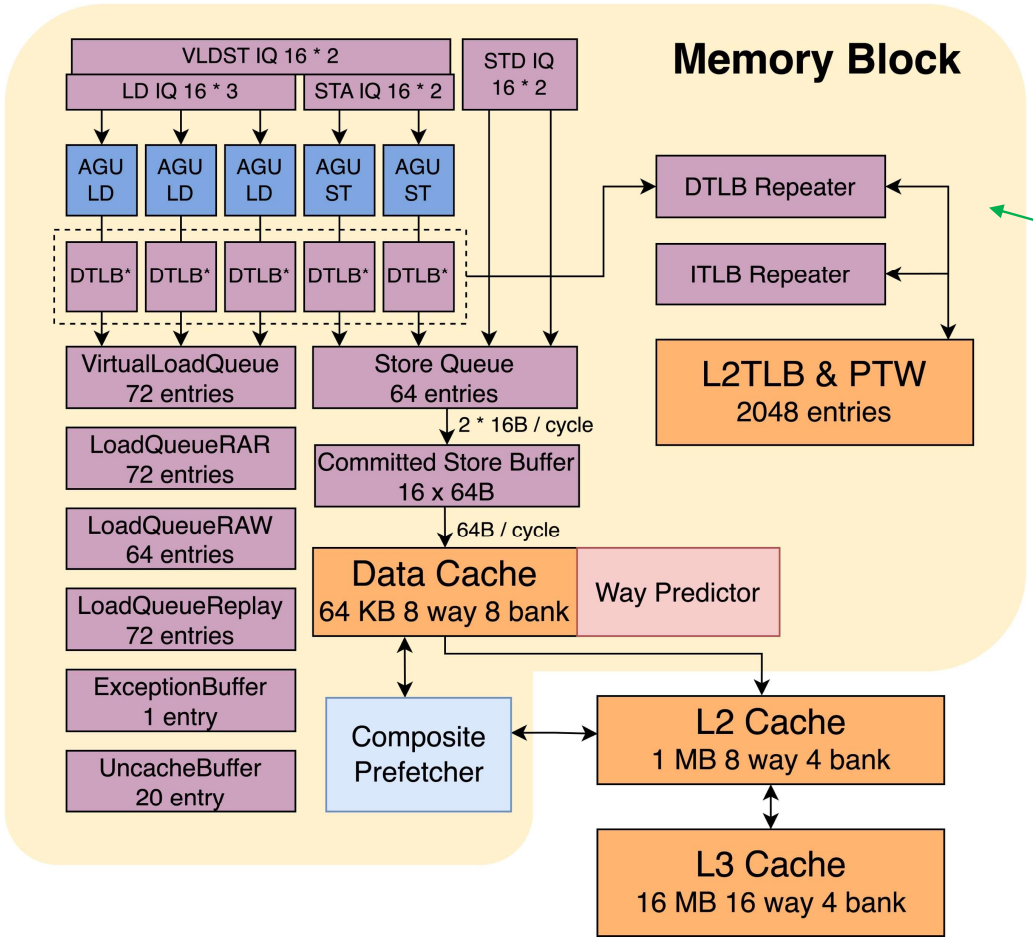
- **4** FPU + **2** FP Div

Vector block

- **4** VPU + **1** Vec Div
- **V1.0** SPEC (VLEN = **128**)



3rd Kunminghu µArch Overview



Load/Store pipeline & queue

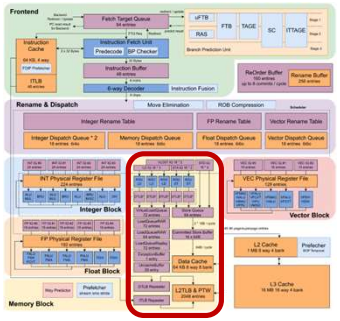
- **3** load pipes, **2** store pipes
- **72** inflight load, **64** inflight store

MMU

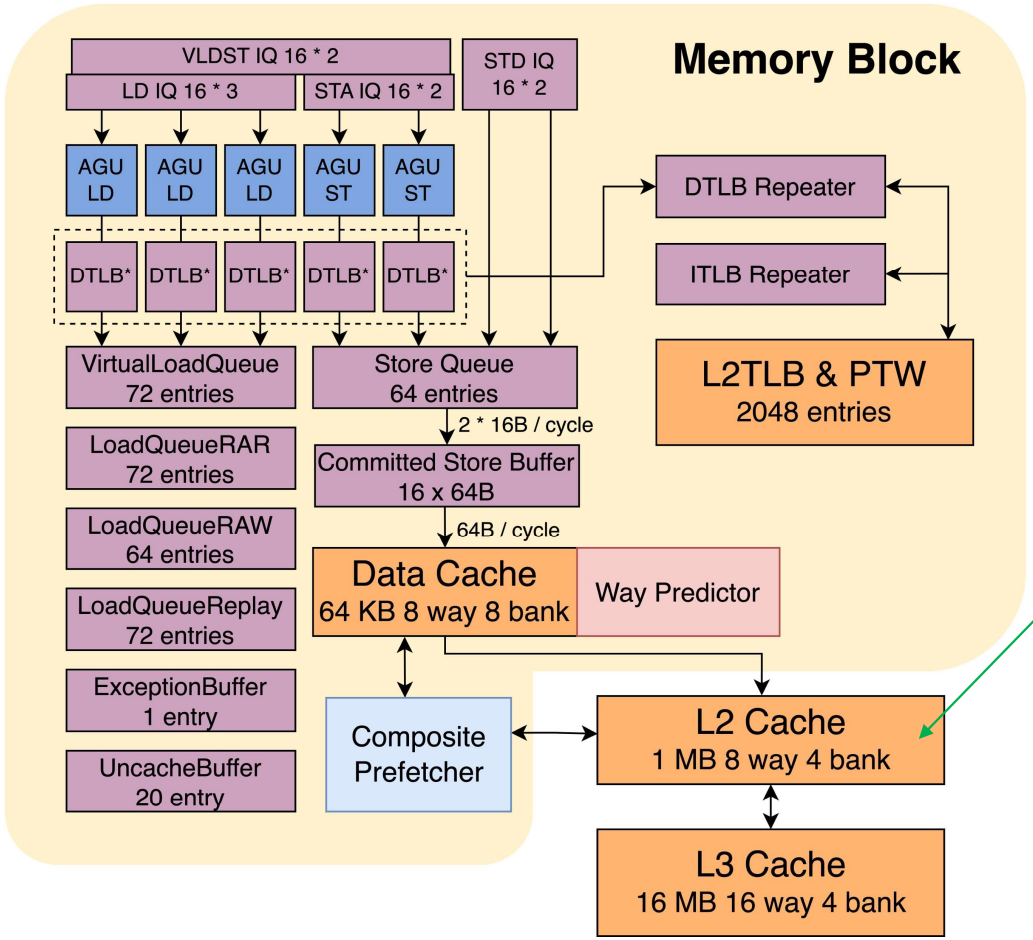
- **48b** virtual/**48b** physical addressing
- **48**-entry L1 DTLB, **2K**-entry L2 TLB
- Up to **7** outstanding page table walks
- Nested walk for virtualization

Data cache

- **64KB 8-way** VIPT (HW anti-aliasing)
- Way predictor for power efficiency
- Composite prefetcher
  - Stream & Stride prefetching
  - SMS & BOP prefetching
  - Temporal prefetching



3rd Kunminghu µArch Overview

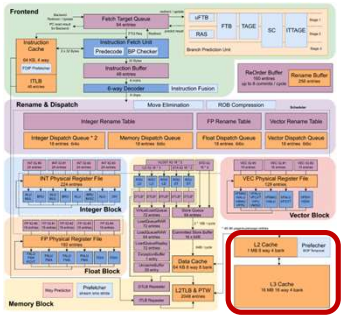


Private L2 cache

- **8-way**, up to **1MB** per core
- Inclusive to D$, non-inclusive to I$
- **64**+ OTs (configurable)
- **10-cycle** load-use latency
- PLRU/RRIP replacement

Shared L3 cache

- **16-way**, up to **16MB**
- Inclusive/Non-inclusive to L2$
- **150**+ OTs (configurable)
- **~30-cycle** load-use latency
- PLRU/RRIP replacement



Performance Evaluation of XiangShan 3rd Kunminghu

- **Method: SPEC CPU checkpoints selected by SimPoint**
  - Compiler: GCC 12 -O3, RV64GCB, jemalloc
  - TileLink bus protocol
  - Cache: 64KB I$/D$ + 1MB L2$ + 16MB L3$
  - Memory modeled by DRAMsim3 DDR4@3200MHz
    - 70ns latency
    - Dual channel, 2x64
- **Evaluation results**

Base score	SPECint 2006	SPECfp 2006
Nanhu@2GHz	**16.94**	**19.42**
Kunminghu*@3GHz	**44.58**	**44.54****

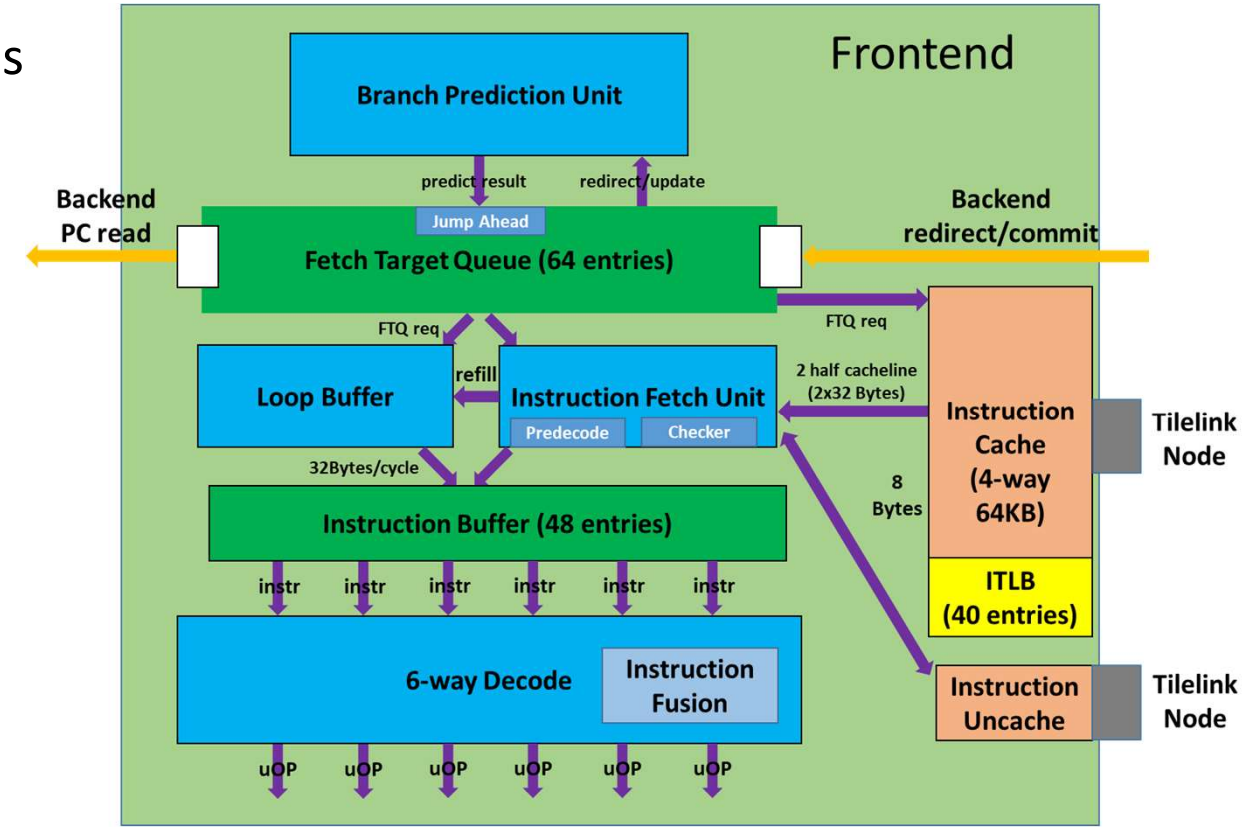
SPECint 2006 est. @ 3GHz		SPECfp 2006 est. @ 3GHz	
400.perlbench	35.91	410.bwaves	66.86
401.bzip2	25.51	416.gamess	40.89
403.gcc	47.87	433.milc	45.17
429.mcf	59.56	434.zeusmp	52.09
445.gobmk	30.31	435.gromacs	33.66
456.hmmer	41.61	436.cactusADM	46.18
458.sjeng	30.52	437.leslie3d	45.90
462.libquantum	122.43	444.namd	28.88
464.h264ref	56.58	447.dealII	73.57
471.omnetpp	41.42	450.soplex	52.63
473.astar	29.30	453.povray	53.39
483.xalancbmk	72.95	454.Calculix	16.38
GEOMEAN	**44.58**	459.GemsFDTD	35.68
SimPoint Based Sampling GCC 12 -O3, RV64GCB, jemalloc DRAMsim3 DDR4 @ 3200MHz		465.tonto	36.68
		470.lbm	91.19
		481.wrf	40.64
		482.sphinx3	49.11
		GEOMEAN	**44.54**

Published by latest XiangShan Biweekly

* Updated in Apirl 2025, XiangShan commitID 6683fc4
** FP performance slightly degraded mainly due to PPA optimisation

Frontend Upgrade Overview

- Decoupled architecture
  - Hiding branch prediction bubbles
  - Guiding instruction prefetch
- Iterative optimization
  - Lower prediction miss rate
  - Larger fetch/prediction width
  - More accurate branch predictor



Frontend Feature 1: TAGE Predictor Fine Tuning

- ❶ #Table & History Length

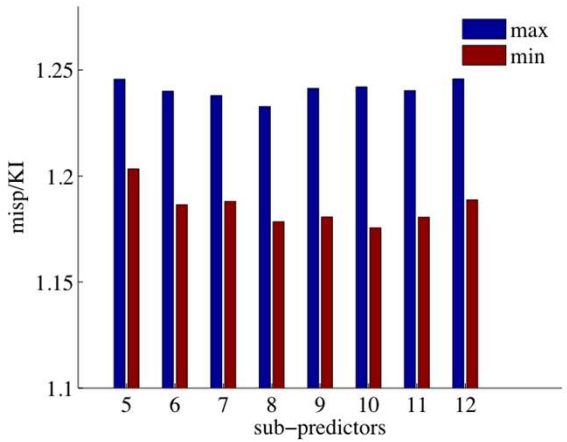
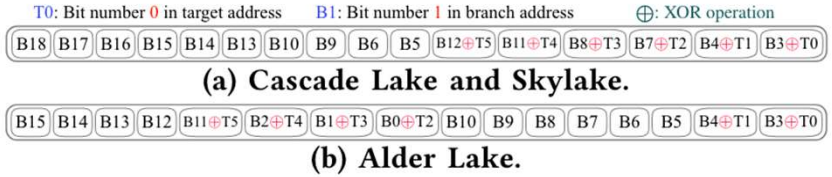


Figure 2: Performance of random parameters

Previous studies have shown that the #tables and history length have a significant effect on TAGE performance, up to 5.64% in MPKI*

* C. Zhou, L. Huang, Z. Li, T. Zhang, and Q. Dou, "Design Space Exploration of TAGE Branch Predictor with Ultra-Small RAM," in *Proceedings of the on Great Lakes Symposium on VLSI 2017*, Banff Alberta Canada: ACM, May 2017, pp. 281–286.

- ❷ Branch History Hashing



Hash algorithm affects performance

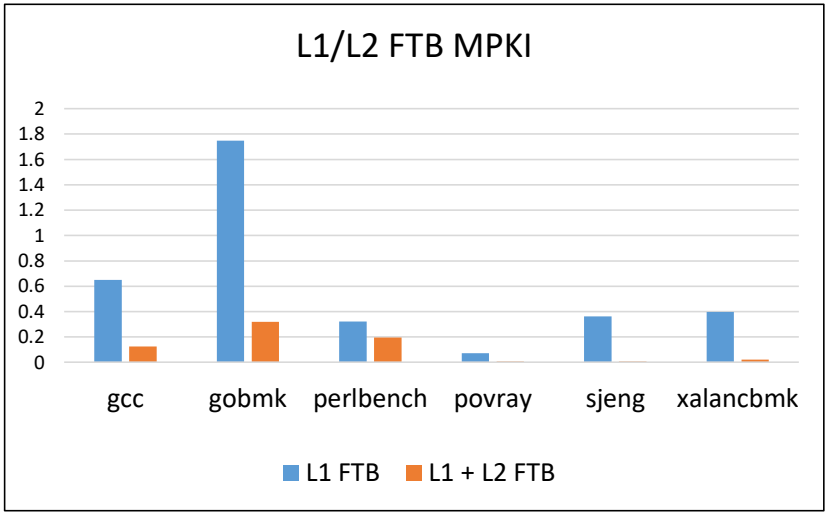
Genetic and particle swarm algorithms are employed to adjust parameters automatically.

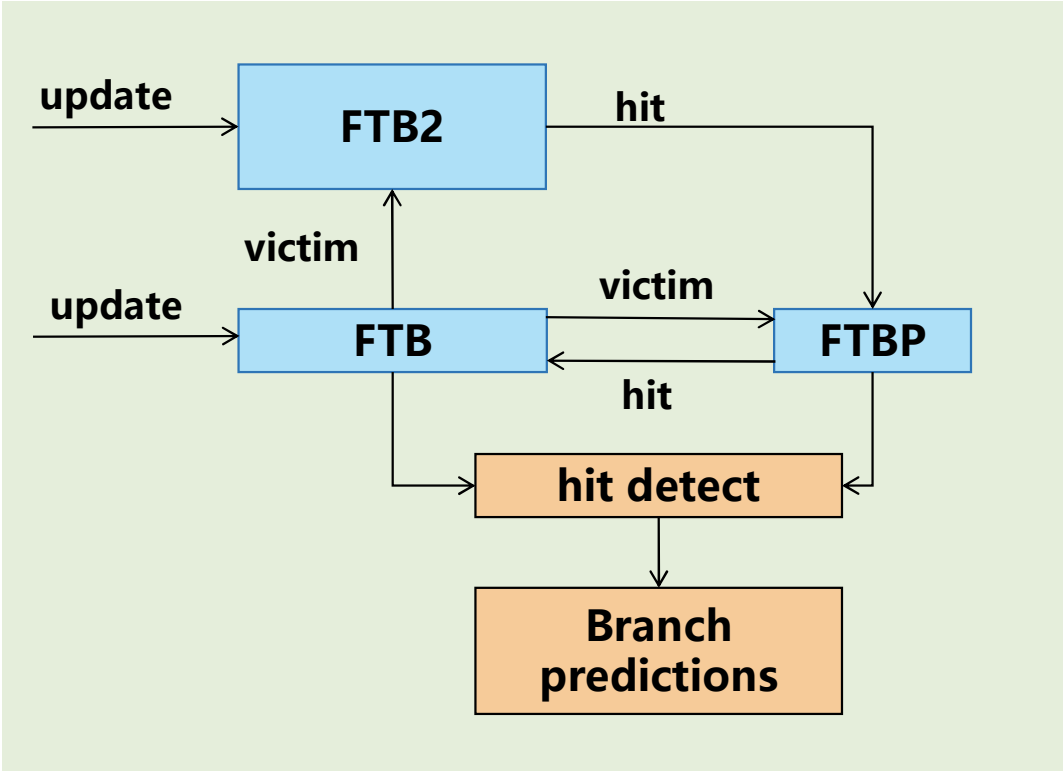
Param of Nanhu → Param tuned: IPC +1.44%

Bonus!

Frontend Feature 2: L2 FTB (BTB)

- Hierarchical FTB
  - Support 4K L2 FTB
  - Semi-exclusive
  - FTBP acts as a buffer between L1 & L2 FTB





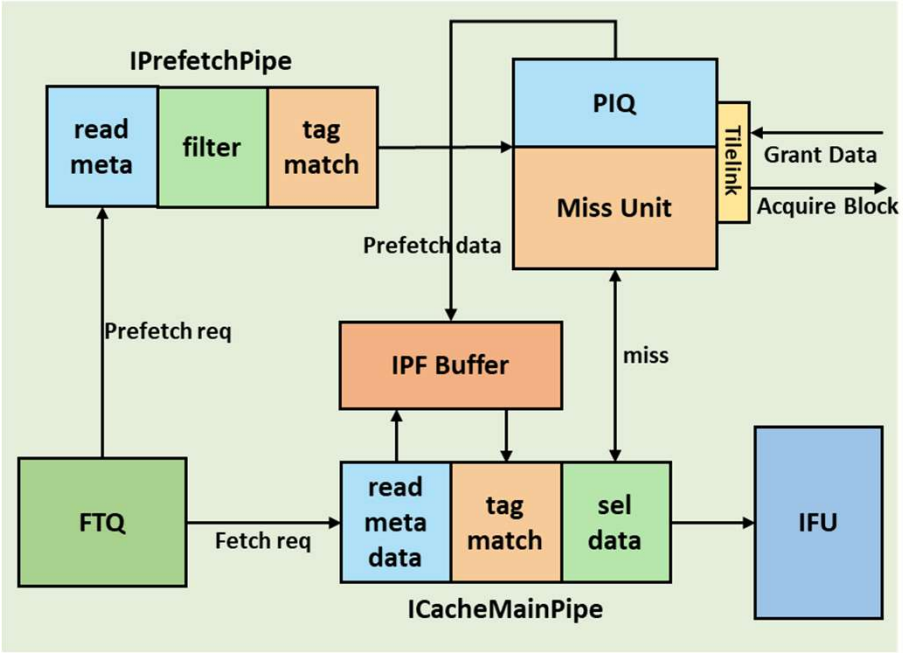
L2 FTB Structure

Frontend Feature 3: ICache Smarter Prefetcher

- Based on decoupled frontend, FDIP algorithm is implemented
- **Prefetcher workflow**

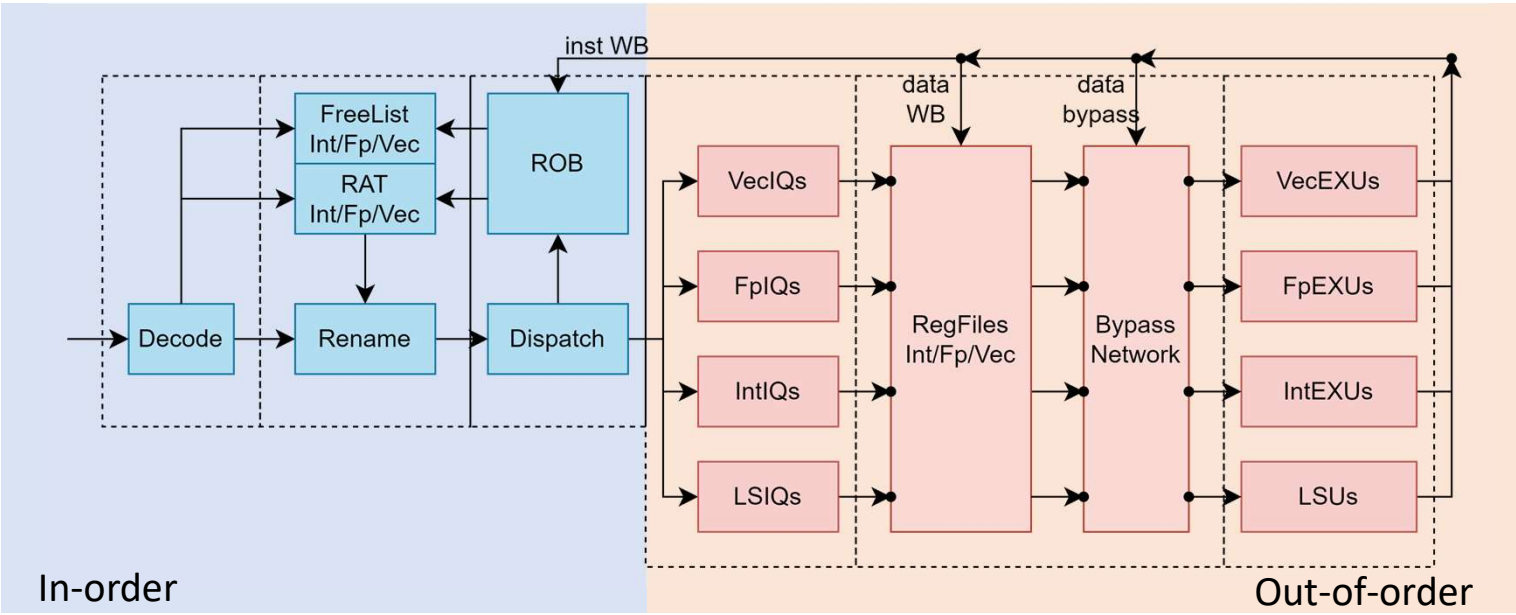
1. Select prefetch request from PF-queue
2. Write refilled data into prefetch buffer
3. Fetch query icache & prefetch buffer parallelly

- **Optimisation compared to nanhu**
  - Prefetch position: L2 Cache -> **L1 ICache**
  - **Dual prefetching pipelines** to enlarge bandwidth



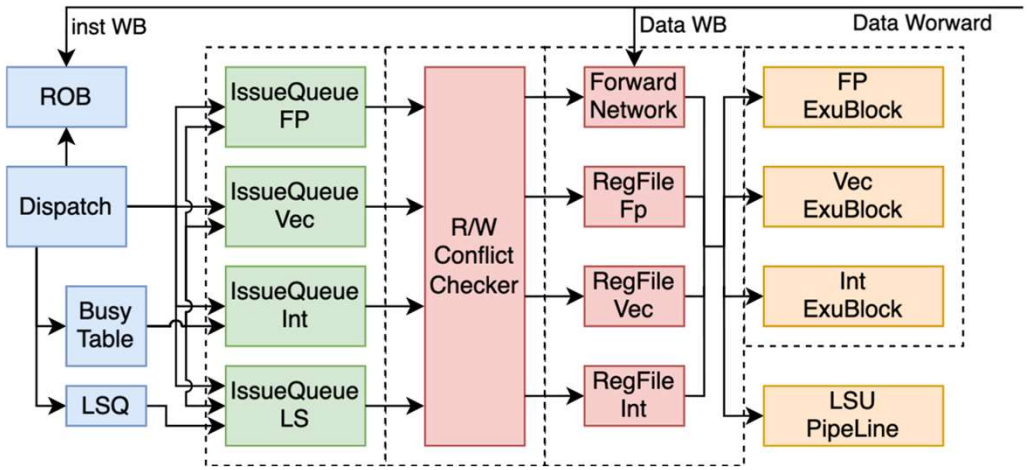
Backend Upgrade Overview

- Read regfile after issue
- Faster µop commit (ROB compression) and redirect (Rename snapshot)
- Vector extension and Hypervisor extension support



Backend Feature 1: Read Regfile after Issue

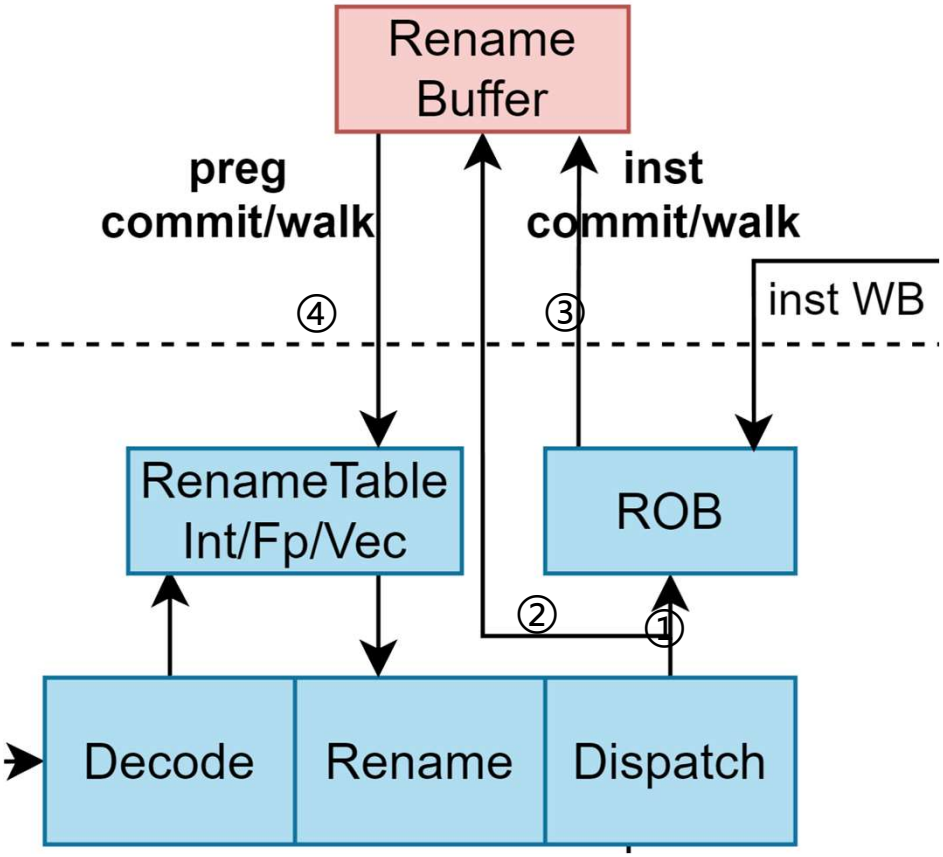
- Advantage
  - Bottleneck is shifted to after µop issue
  - Reduce RS storage to save SRAM
  - Reduce latency and increase RS capacity
- Optimization
  - Efficient arbitration of regfile reading ports
  - Accurate speculative wakeup and cancelling



Read regfile after issue

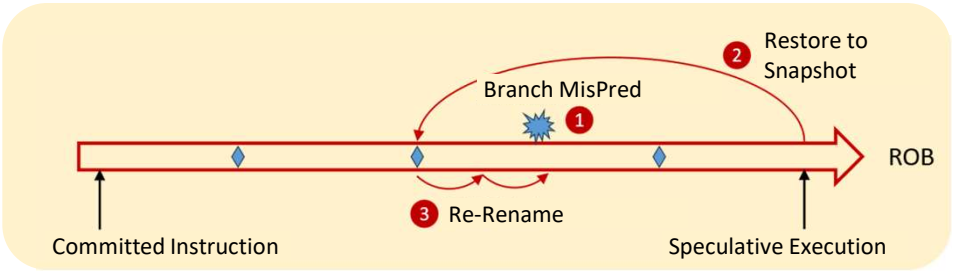
Backend Feature 2: Decouple µop & Regfile Commit

- New Rename Buffer (**RAB**) design: record regfile changing history
  - ① Save Inst in ROB after dispatch
  - ② Save regfile mapping info to Rename Buffer
  - ③ On Inst commit/redirect, wake RAB up
  - ④ RAB is responsible to update Rename Table
- Benefits
  - Faster ROB entry releasing
  - Optimize timing
  - Support µop split of vector instructions
  - Support ROB compression

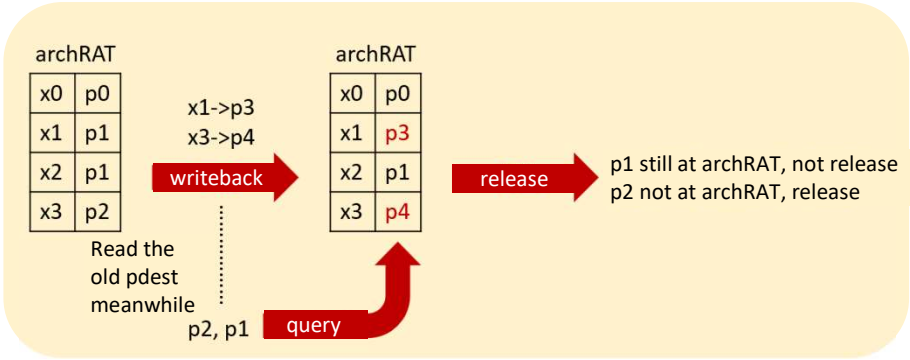


Backend Feature 3: Register Renaming Optimizations

- ① Rename Snapshot: Reduce The Cost of Backend Redirect
- ② Move Elimination Based on RAT: Optimize Area and Power
- ③ ROB Compression: Reduce ROB Consuming and Enlarge OoO Windows

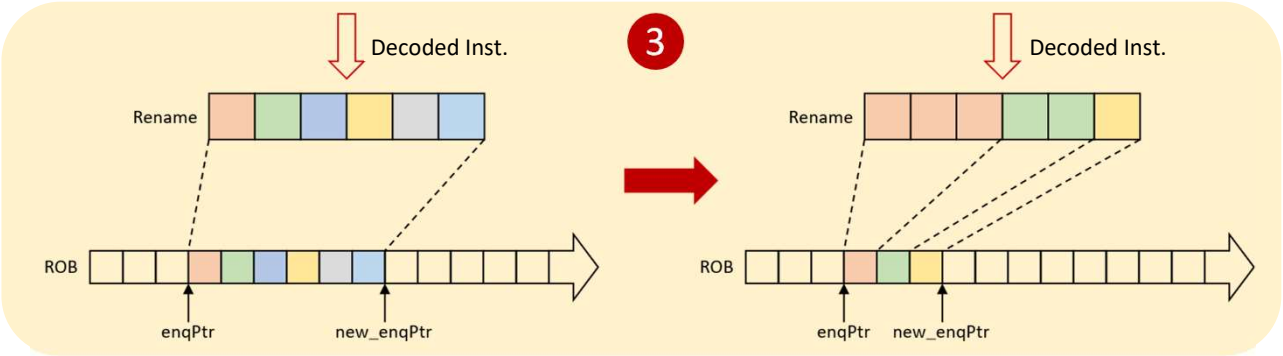


1



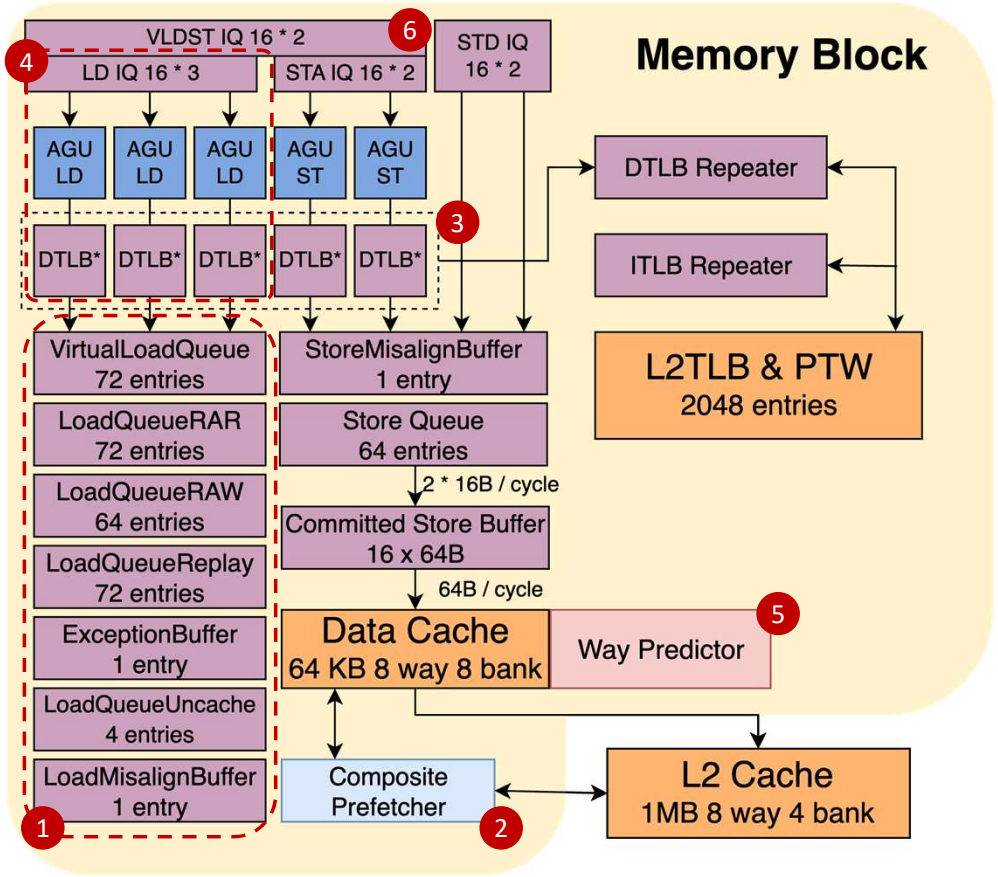
2

3



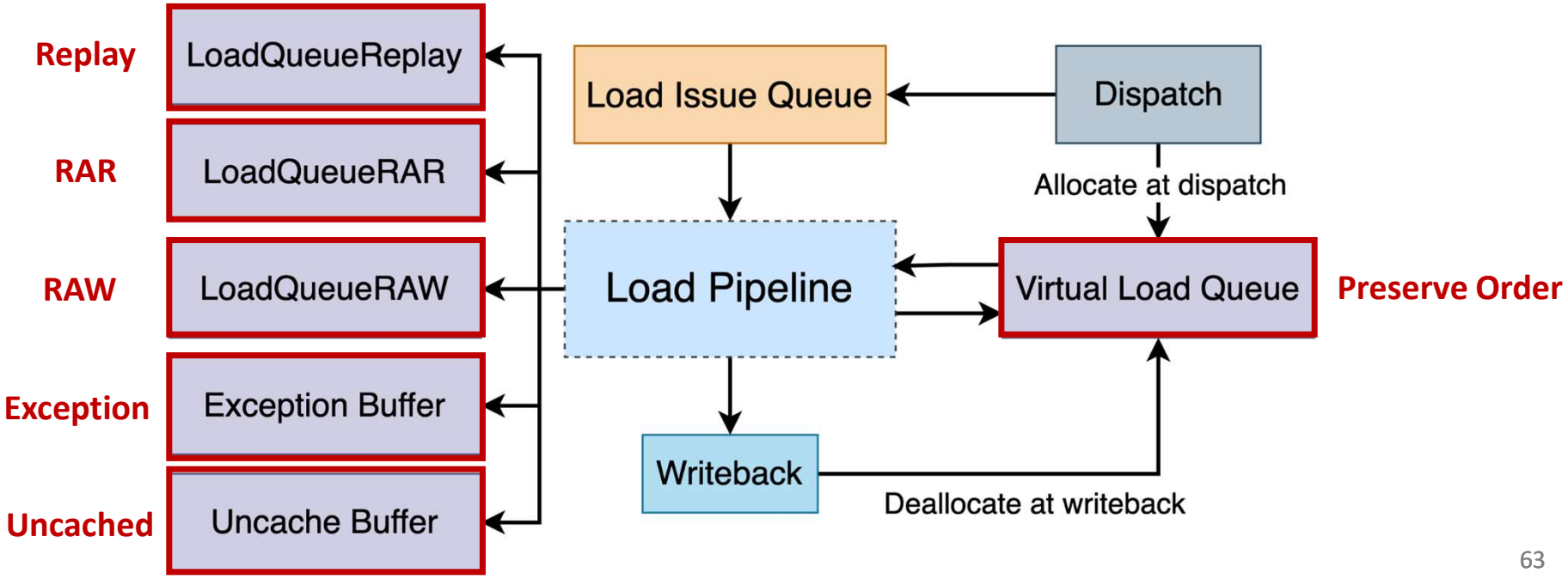
MemBlock Upgrade Overview

- Optimization compared to NH
  - ① Divide load queue
  - ② Prefetch to L1D
  - ③ Increase effective TLB capacity
  - ④ Enlarge load bandwidth
  - ⑤ Implement way predictor
  - ⑥ Vector LSU



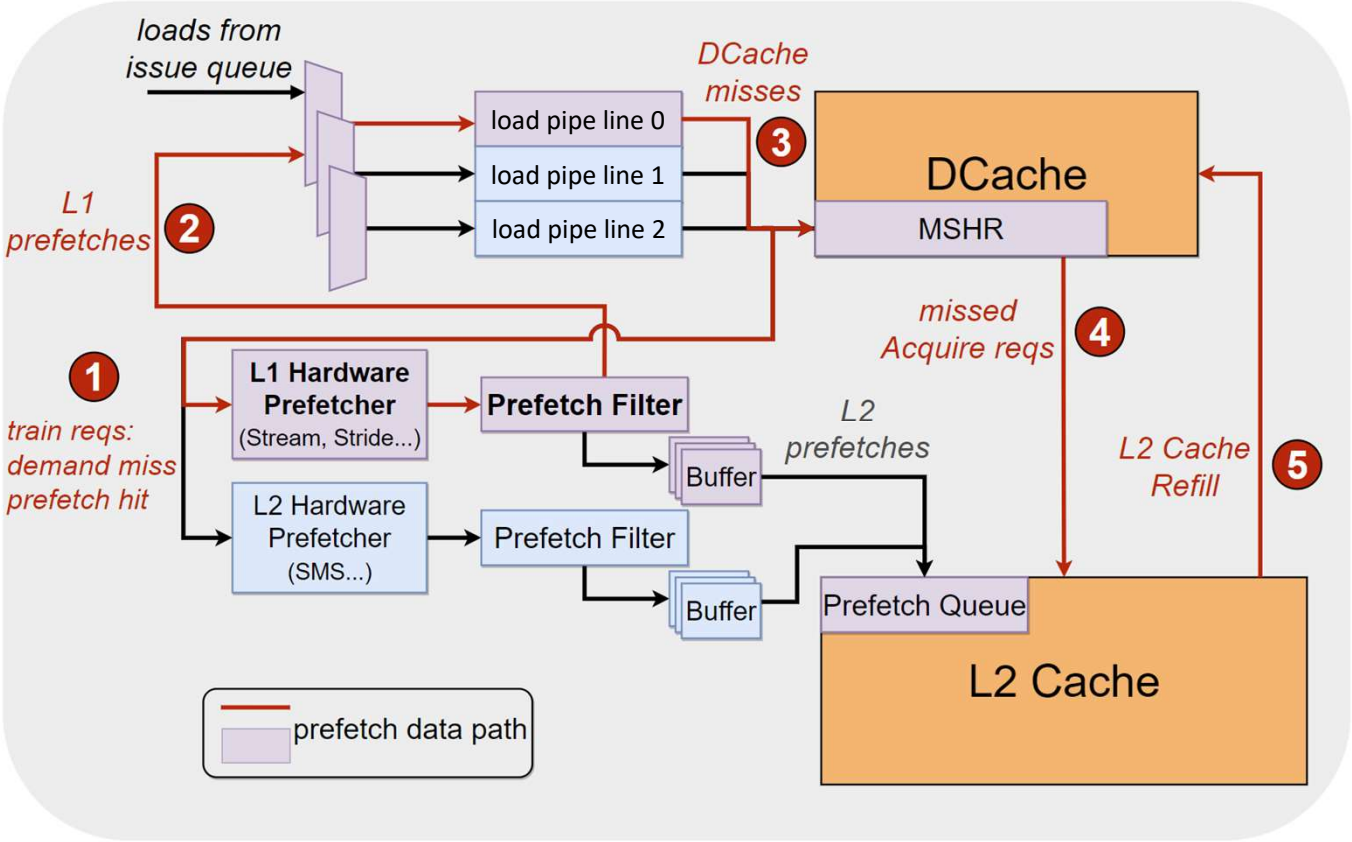
MemBlock Feature 1: Split Load Queue

- Split load queue **based on different responsibilities**
  - Logic simpler, timing friendly and area acceptable
  - **Early commit**: load instruction can be retired sequentially from lq after write-back



MemBlock Feature 2: L1D Prefetch

- L1D prefetch workflow
  - Training pattern detection
  - **Reuse load pipelines**
  - Fetch data from D$ MSHR
- Prefetch algorithm
  - **Stream[1]**
    - (x,x+1,x+2,x+3...)
  - **Stride[2]**
    - (x,x+n,x+2n,x+3n...)

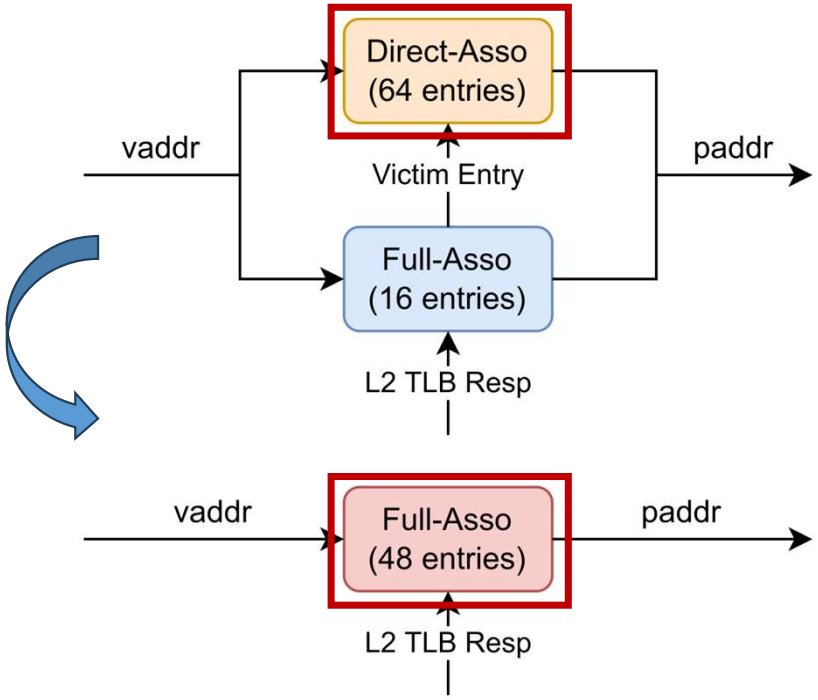


[1] S. Srinath, O. Mutlu, H. Kim and Y. N. Patt, "Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers," 2007 IEEE 13th International Symposium on High Performance Computer Architecture, Scottsdale, AZ, USA, 2007

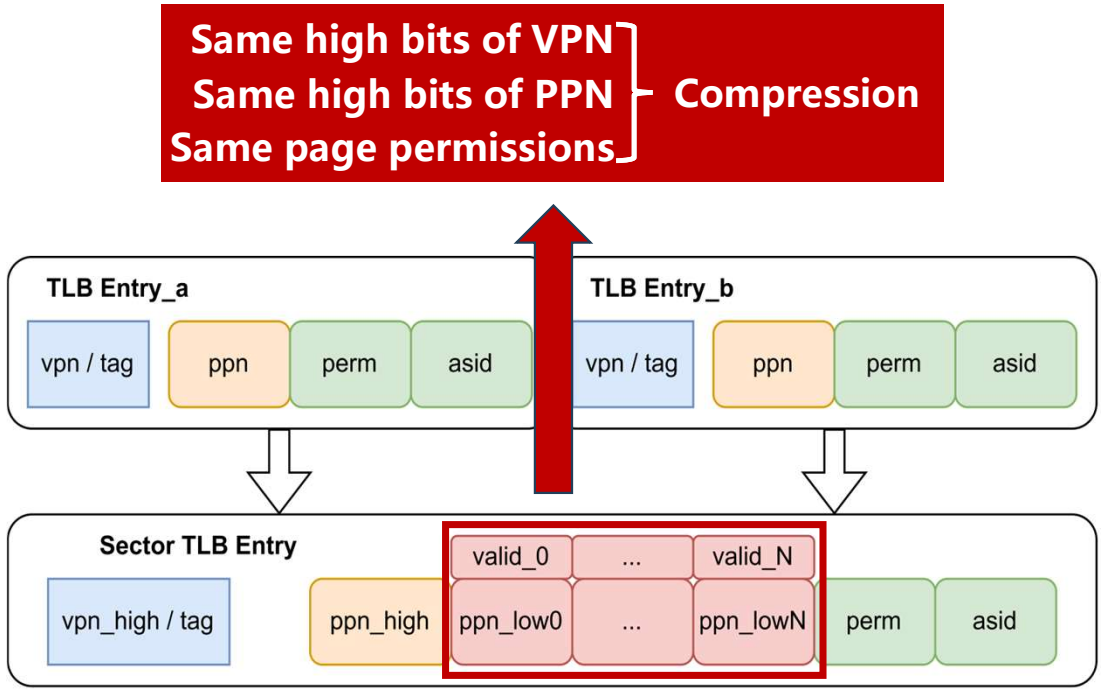
[2] J. -L. Baer and T. -F. Chen, "An effective on-chip preloading scheme to reduce data access penalty," Supercomputing '91:Proceedings of the 1991 ACM/IEEE Conference on Supercomputing, Albuquerque, NM, USA, 1991

MemBlock Feature 3: Increase effective TLB capacity

- 16 fully associative entry + 64 direct mapping entry → **48 fully associative entry**
- Support **TLB compression**, merging contiguous page table entries



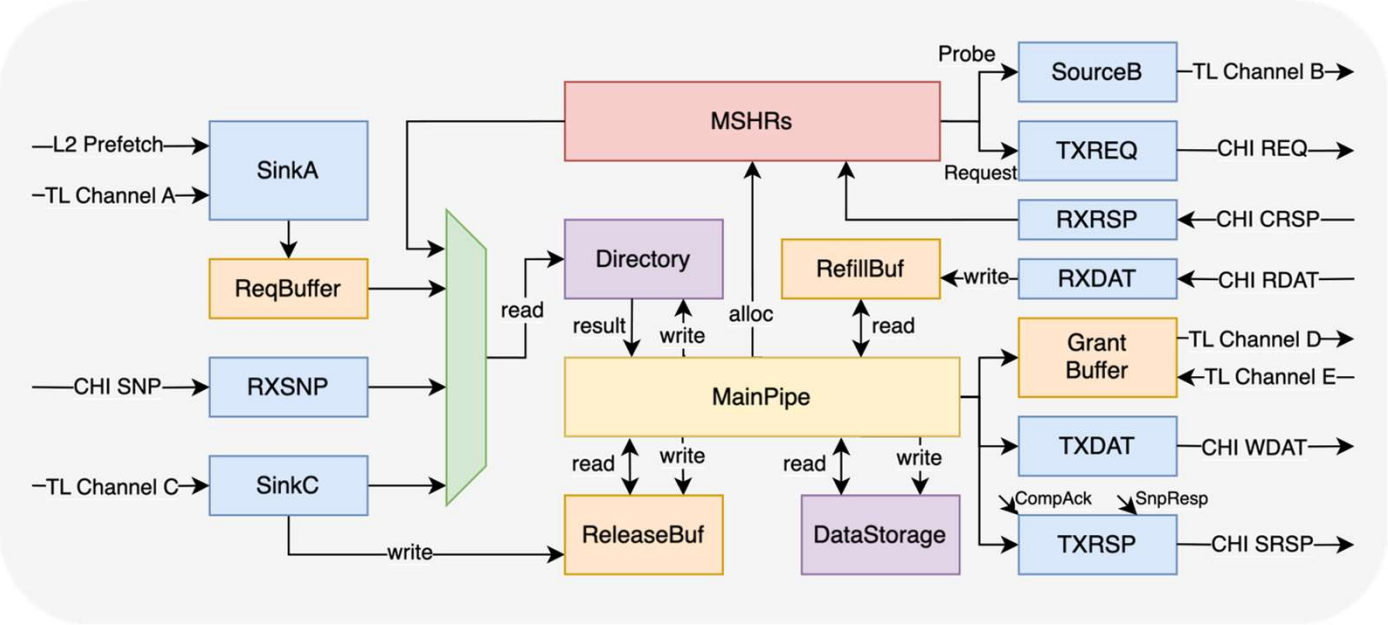
DTLB organization in Nanhu / Kunminghu



TLB compression

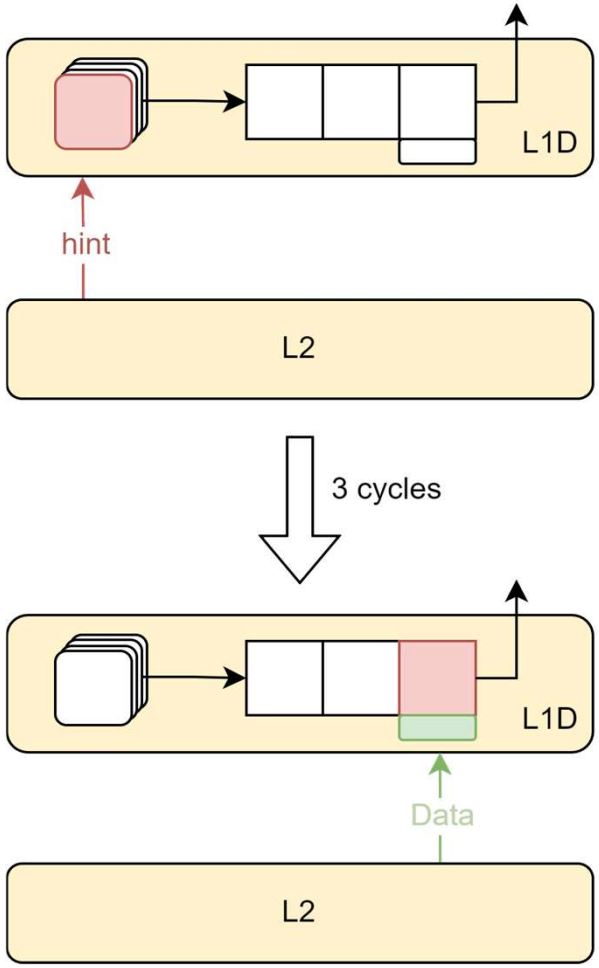
L2 Cache Upgrade Overview

- Multiple concurrent pipelines → Non-blocking main pipeline
- L1-L2 Refill wake-up collaboratively
- Eliminate transaction serialization in the same set & Request merge
- Supported protocols
  - CHI Issue E.b
  - CHI Issue B
  - CHI Issue C
  - TileLink



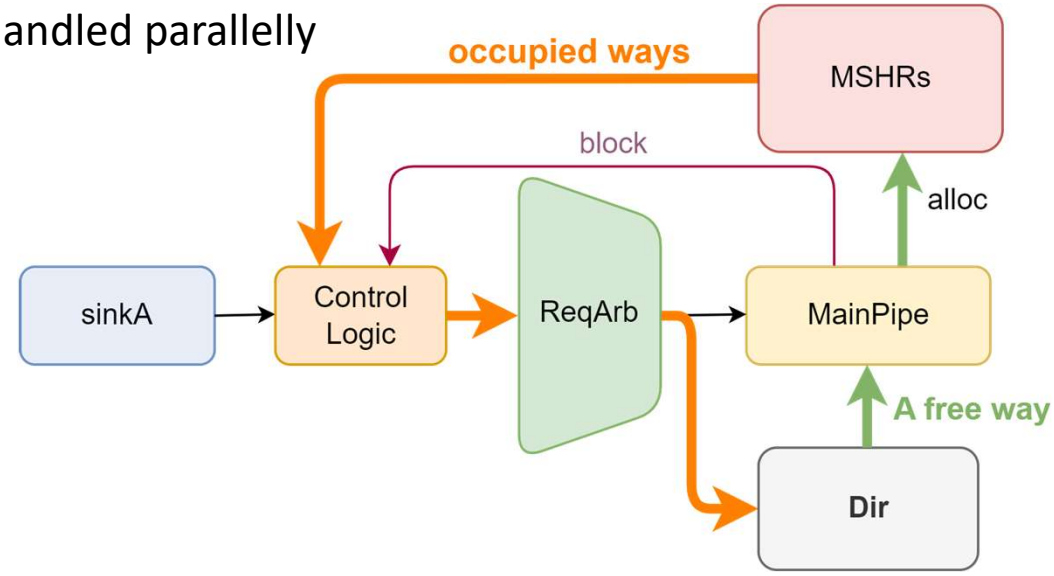
L2 Cache Feature 1: L1-L2 Collaborative Optimization

- Observation：
  - Interval between **load wake-up** and **use-refill-data** is 3 cycles
  - There’s chance to hint LSU to replay before refill
- New Design：
  - Let L2 give **Hint signal in advance**
  - Speedup the wakeup and replay of load replay queue
- Implementation：
  - Set up monitor to track info from main pipeline
  - Need to calculate the exact Hint timing
  - Send hint signal to L1 3 cycles before refill



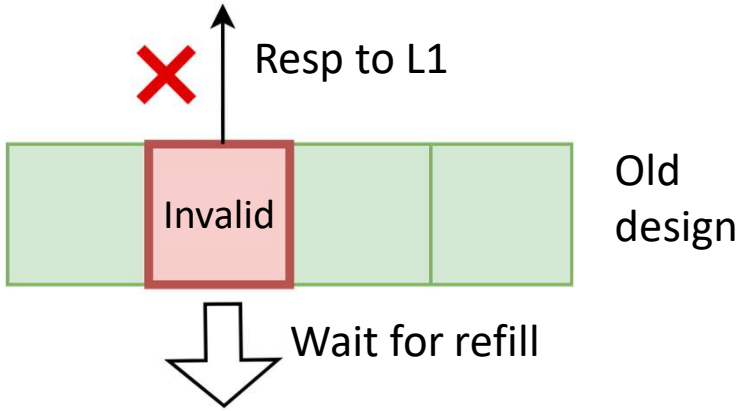
L2 Cache Feature 2: Eliminate Txns Serialization in same set

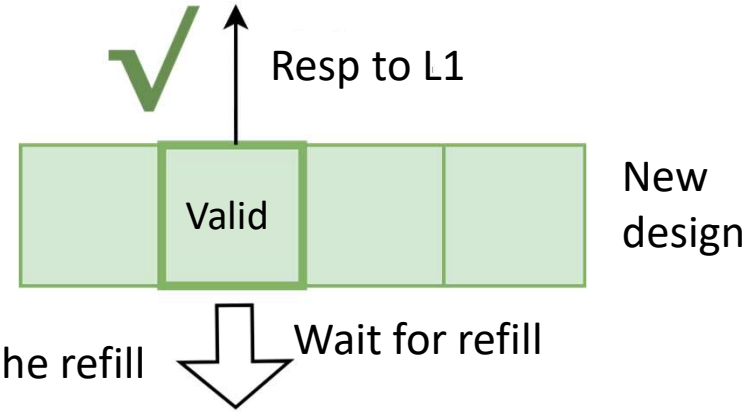
- Background: Txns to **same sets** were handled serially, reduces parallelism
- Difficulty: Same-set txns can affect each other, due to replacement
- New Design
  - Txns of **same-set but different-addr** can be handled parallelly
  - Txns of same-addr are handled serially
- Implementation
  - Record occupied ways in active MSHRs
  - **Assign a free way** to the new request



L2 Cache Feature 3: Evict on Refill

- Observation：
  - On L1 acquire, L2 chooses a way for replacement
  - But this way is **occupied until refill**, unable to serve L1
- Improvement：
  - New design chooses replaced-way **after** data is refilled from L3
  - So, it can still serve L1 when waiting for refill
- Implementation：
  - Transfer it to L3 on acquire miss, make replacer untouched
  - Wait for L3 to refill
  - **Read directory again** and assign one way when MSHR handles the refill





XiangShan: Open-Source High-Performance Processor

XiangShan Project

- **Embraces innovative agile development workflow**
- **Fills the gap in open-source high-performance processors**
- **Addresses the needs from both industry and academia**

- **1st generation: YQH (Yanqihu) & 2nd generation: NH (Nanhu)**
- **3rd generation: KMH (Kunminghu)**
  - **Support RVA23 profile and L2 Cache with CHI protocol**
  - **3GHz @7nm, estimated SPEC CPU2006 45@3GHz**
- **Open-sourced at https://github.com/OpenXiangShan/XiangShan**
- **NEXT PART: Hands-on Development**
  - Scan the QR to get the env introduction and slides

What we will cover in this tutorial

- Introduce of XiangShan (11:30 – 12:00, 30 minutes)
- CPU Microarchitecture (12:00 – 12:30, 30 minutes)
- **Development workflows (12:30 - 13:00, 30 minutes)**
  - *Introduction of their usages – How to develop on XiangShan with MinJie*
  - Simulation and Debugging
  - Debug Demo

In this tutorial, we will

- Provide access to cloud servers
- Prepare the XiangShan development environment for you
- Go through the development workflows including:

Simulation

- Server login
- Environment
- RTL Generation
- RTL Simulation

……

Func. Verification

- Nexus-AM
- NEMU
- Difftest
- TL-Test

……

- ***Required: laptop with browser or SSH***

Demo Instructions

- **Shell commands** are highlighted in **brown box** with prefix $

```
$ echo "Hello, XiangShan"
$ echo "Have a nice day"
```

- **Description and notes** are presented in **grey box** with prefix #

```
# Please prepare a laptop with an SSH client.
# Next, let's start the demo session !
```

Let's Start!

Prerequisites

- **Login to the provided cloud server**
  - **Web Browser (VS Code Server)**

    https://t.xiangshan.cc

  - **SSH Access**

```
$ ssh guest@t.xiangshan.cc
# Password: xiangshan-2025
$ guest@xiangshan-tutorial:~$ pwd
/home/guest
```

For **offline users**, please refer to https://github.com/OpenXiangShan/xs-env/tree/tutorial-2025

Prerequisites

```
# Copy tutorial environment to your dir based on your name
$ cp -r /opt/xs-env ~/<YOUR_NAME>

# Enter your dir
$ cd ~/<YOUR_NAME>

# Set up environment variables
# DO IT AGAIN when opening a new terminal
$ source env.sh

# SET XS_PROJECT_ROOT:                    /home/guest/YOUR_NAME
# SET NOOP_HOME (XiangShan RTL Home):     $XS_PROJECT_ROOT/XiangShan
# SET NEMU_HOME:                          $XS_PROJECT_ROOT/NEMU
# SET AM_HOME:                            $XS_PROJECT_ROOT/nexus-am
# SET TLT_HOME:                           $XS_PROJECT_ROOT/tl-test-new
# SET gem5_home:                          $XS_PROJECT_ROOT/gem5
```

Chisel Compilation

- **Compile RTL and build simulator with Verilator (*NOT today*)**

```
$ make emu -j4
Options:
    CONFIG=MinimalConfig      Configuration of XiangShan {MinimalConfig, DefaultConfig, ...}
    // EMU_THREADS=4          Simulation threads
    // EMU_TRACE=1            Enable waveform dump
    // WITH_DRAMSIM=1         Enable DRAMSim3 for DRAM simulation
    // WITH_CHISELDB=1        Enable ChiselDB feature
    // WITH_CONSTANTIN=1      Enable Constantin feature
```

- **Compilation might take ~20 mins**

Run RTL Simulation with Verilator

- **After building, we can run the simulator**

```
# run pre-built simulator
$ cd tutorial/p1-basic-func
$ ./emu -i hello.bin --no-diff 2>hello.err

# Some key options:
    -i                              # Workload to run
    -C / -I                         # Max cycles  / Max Insts
    --diff=PATH or --no-diff        # Path of Reference Model or disable difftest
                                    # Ref model can be SPIKE or NEMU
```

Great!

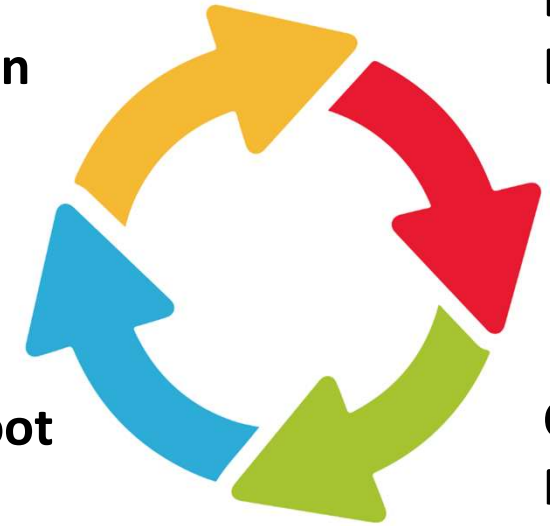
We have learned the basic simulation process of Xiangshan.

Once we get the RTL simulator ready, what's next?

How can we

- **generate desired workloads**
- **find and fix functional bugs**
- **do performance analysis**
- **do research on micro-architecture**

MinJie Functional Verification



Nexus-AM

Test Generation

Bug Detection

Difftest

NEMU

Waveform

ChiselDB

Troubleshoot & Fix bugs

Context Preservation

LightSSS

Nexus-AM: Bare Metal Runtime

- **Purpose**
  - Generate workloads **agilely** without an OS
  - Provide runtime framework for **bare metal machines** like XiangShan
- **We present a bare metal runtime called Nexus-AM (Abstract Machine)**
  - Light-weight, easy to use
  - Implement basic **system call** interfaces and exception handlers

Nexus-AM

- **Hands-on:** build Coremark workload
- **Showcase**
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p1-basic-func**

```
$ bash build-am.sh
```

```
# cd $AM_HOME/apps/coremark
# make ARCH=riscv64-xs \
  ITERATIONS=1 LINUX_GNU_TOOLCHAIN=1 TOTAL_DATA_SIZE=400
# ls -l build
#     coremark-riscv64-xs.bin    Binary image of the program
#     coremark-riscv64-xs.elf    ELF of the program
#     coremark-riscv64-xs.txt    Disassembly of the program
```

MinJie Functional Verification

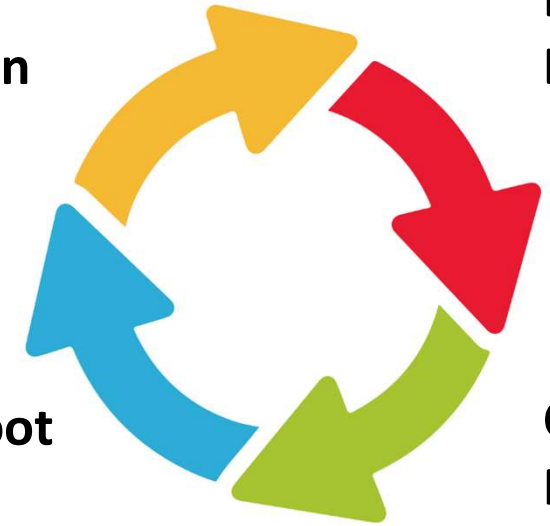
Nexus-AM

Test Generation

Bug Detection

Difftest

NEMU



Waveform

ChiselDB

Troubleshoot & Fix bugs

Context Preservation

LightSSS

NEMU: ISA REF

- **Purpose**
  - Golden model for verification
  - Simple yet high performance
- **We present NEMU**
  - An **instruction set simulator**, similar to Spike
  - Through **optimization techniques**, achieve performance similar to QEMU.
  - **A set of APIs** is provided to assist XiangShan in comparing and verifying the **architectural states**

NEMU

- **Hands-on:** build NEMU interpreter and dynamic link file
- **Showcase**
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p1-basic-func**

```
$ bash build-nemu.sh
```

```
# cd $NEMU_HOME
# make clean
# make riscv64-xs_defconfig
# make -j      build NEMU as the bare metal machine, can run the Coremark from the previous step

# make clean-softfloat
# make riscv64-xs-ref_defconfig
# make -j      build NEMU as the reference model for XiangShan
```

NEMU

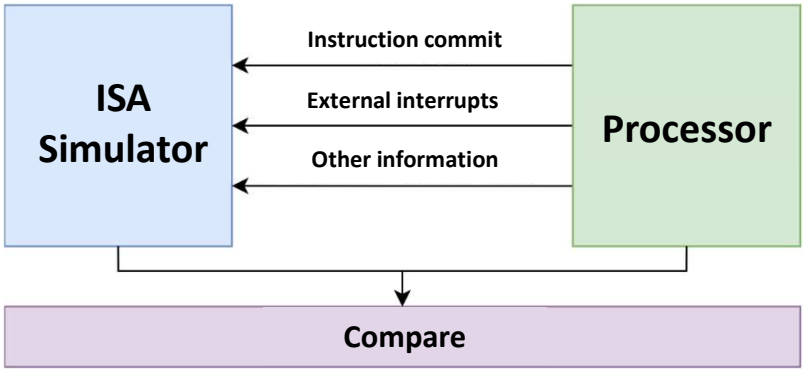
- **Hands-on:** run Coremark workload on NEMU
- **Showcase**
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p1-basic-func**

```
$ bash run-nemu.sh
```

```
# cd $NEMU_HOME                                          run in batch mode, faster
# ./build/riscv64-nemu-interpreter -b \                  set workspace to Coremark
      $AM_HOME/apps/coremark/build/coremark-1-iteration-riscv64-xs.bin
```

Difftest: ISA Co-simulation Framework

- **Basic flows**
  - Instructions commit/other states update
  - The simulator executes the same instructions
  - Compare the architectural states between DUT and REF
  - Abort or continue
- **Provided as verification infrastructure for processors**
  - APIs for HDLs such as Chisel/Verilog
  - RTL simulators such as Verilator, VCS
  - RISC-V Instruction Set Simulators, such as **NEMU**, **Spike**
- **SMP-Difftest: co-simulation on an SMP processor**
  - SMP Linux kernel and multi-threading programs
  - Online checking of cache coherency and memory consistency



Basic architecture of DiffTest

```
while (1) {
    icnt = cpu_step();
    nemu_step(icnt);
    r1s = cpu_getregs();
    r2s = nemu_getregs();
    if (r1s != r2s)
        abort();
}
```

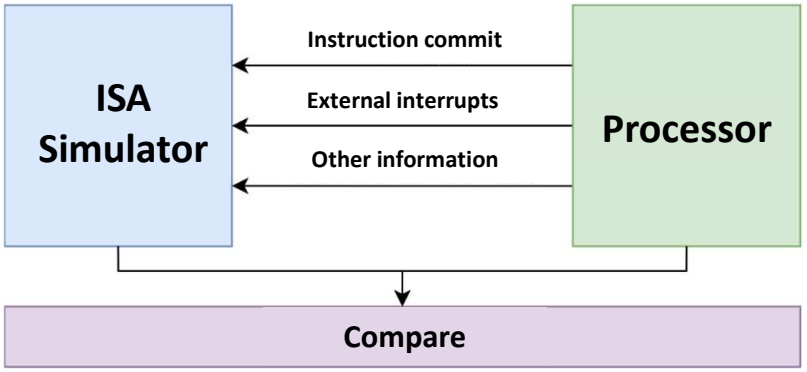
Online checking

Difftest: ISA Co-simulation Framework

- **Basic flows**
  - Instructions commit/other states update
  - The simulator executes the same instructions
  - Compare the architectural states between DUT and REF
  - Abort or continue

- **Provided as verification infrastructure for processors**
  - APIs for HDLs such as Chisel/Verilog
  - RTL simulators such as Verilator, VCS
  - RISC-V Instruction Set Simulators, such as **NEMU**, **Spike**

- **SMP-Difftest: co-simulation on an SMP processor**
  - SMP Linux kernel and multi-threading programs
  - Online checking of cache coherency and memory consistency



Basic architecture of DiffTest

```
while (1) {
    icnt = cpu_step();
    nemu_step(icnt);
    r1s = cpu_getregs();
    r2s = nemu_getregs();
    if (r1s != r2s)
        abort();
}
```

Online checking

Difftest

- **Hands-on:** run Coremark workload on XiangShan, difftest with NEMU
  - **Not today**
- **Showcase**

```
# Running Coremark takes about 3 minutes
$ bash run-emu.sh
```

```
# ./emu -i \    use coremark.bin
$AM_HOME/apps/coremark/build/coremark-1-iteration-riscv64-xs.bin
--diff $NEMU_HOME/build/riscv64-nemu-interpreter-so \      difftest with NEMU
2>perf.out   \     redirect stderr to file
```

Difftest

```
./emu -i $AM_HOME/apps/coremark/build/coremark-1-iteration-riscv64-xs.bin --diff $NEMU_HOME/build/riscv64-nemu-interpreter-so 2>perf.err
emu compiled at Feb 21 2025, 10:49:46
Using simulated 32768B flash
Using simulated 8386560MB RAM
The image is /nfs/home/mayuexiao/tutorial/xs-env/nexus-am/apps/coremark/build/coremark-1-iteration-riscv64-xs.bin
The reference model is /nfs/home/mayuexiao/tutorial/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
Running CoreMark for 1 iterations
2K performance run parameters for coremark.
CoreMark Size    : 666
Total time (ms)  : 1931
Iterations       : 1
Compiler version : GCC11.4.0
seedcrc          : 0xe9f5
[0]crclist       : 0xe714
[0]crcmatrix     : 0x1fd7
[0]crcstate      : 0x8e3a
[0]crcfinal      : 0xe714
Finished in 1931 ms.
==================================================
CoreMark Iterations/Sec 517.87
Core 0: HIT GOOD TRAP at pc = 0x80001de2
Core-0 instrCnt = 379,235, cycleCnt = 252,495, IPC = 1.501951
Seed=0 Guest cycle spent: 252,499 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 164,455ms
```

Difftest

- **Hands-on:**
  - We injected a bug in a pre-built XiangShan processor
  - Now, let's trigger it by Difftest
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p1-basic-func**
- **Showcase**

```
$ bash run-emu-diff.sh
```

```
# ./emu-bug \
  -i $AM_HOME/apps/coremark/build/coremark-1-teration-riscv64-xs.bin \
  --diff $NEMU_HOME/build/riscv64-nemu-interpreter-so    # difftest with NEMU
```

Difftest

- Difftest will report the error once failed
  - Dump info like PC, GPRs, etc.

```
============== Commit Group Trace (Core 0) ==============
commit group [00]: pc 0080001a6e cmtcnt 4
commit group [01]: pc 0080001a7a cmtcnt 1
commit group [02]: pc 0080001a7c cmtcnt 2
commit group [03]: pc 008000167a cmtcnt 2
commit group [04]: pc 008000166c cmtcnt 7 <--
commit group [05]: pc 00800015f4 cmtcnt 2
commit group [06]: pc 00800015f8 cmtcnt 1
commit group [07]: pc 00800015fa cmtcnt 2
commit group [08]: pc 0080001600 cmtcnt 1
commit group [09]: pc 008000160e cmtcnt 3
commit group [10]: pc 0080001614 cmtcnt 1
commit group [11]: pc 008000162a cmtcnt 1
commit group [12]: pc 008000162e cmtcnt 4
commit group [13]: pc 008000163e cmtcnt 10
commit group [14]: pc 0080001658 cmtcnt 7
commit group [15]: pc 008000166a cmtcnt 6
```

```
==============  REF Regs  ==============
---------------- Intger Registers ----------------
  $0: 0x0000000000000000   ra: 0x00000000800015f0   sp: 0x000000008000cef0   gp: 0x0000000000000000
  tp: 0x0000000000000000   t0: 0x0000000000000000   t1: 0x0000000000000001   t2: 0x0000000000000009
  s0: 0x0000000000000000   s1: 0x0000000000000000   a0: 0x000000000000029a   a1: 0x000000008000cf08
  a2: 0x0000000000000002   a3: 0x000000000000029a   a4: 0x0000000000000002   a5: 0x0000000000000007
  a6: 0x0000000000000002   a7: 0x0000000000000003   s2: 0x0000000000000000   s3: 0x0000000000000000
  s4: 0x0000000000000000   s5: 0x0000000000000000   s6: 0x0000000000000000   s7: 0x0000000000000000
  s8: 0x0000000000000000   s9: 0x0000000000000000  s10: 0x0000000000000000  s11: 0x0000000000000000
  t3: 0x00000000800039a0   t4: 0x0000000000000001   t5: 0x000000008000cd91   t6: 0x0000000000000031
---------------- Float Registers ----------------
 ft0: 0xffffffff00000000  ft1: 0xffffffff00000000  ft2: 0xffffffff00000000  ft3: 0xffffffff00000000
 ft4: 0xffffffff00000000  ft5: 0xffffffff00000000  ft6: 0xffffffff00000000  ft7: 0xffffffff00000000
 fs0: 0xffffffff00000000  fs1: 0xffffffff00000000  fa0: 0xffffffff00000000  fa1: 0xffffffff00000000
 fa2: 0xffffffff00000000  fa3: 0xffffffff00000000  fa4: 0xffffffff00000000  fa5: 0xffffffff00000000
 fa6: 0xffffffff00000000  fa7: 0xffffffff00000000  fs2: 0xffffffff00000000  fs3: 0xffffffff00000000
 fs4: 0xffffffff00000000  fs5: 0xffffffff00000000  fs6: 0xffffffff00000000  fs7: 0xffffffff00000000
 fs8: 0xffffffff00000000  fs9: 0xffffffff00000000 fs10: 0xffffffff00000000 fs11: 0xffffffff00000000
 ft8: 0xffffffff00000000  ft9: 0xffffffff00000000 ft10: 0xffffffff00000000 ft11: 0xffffffff00000000
 fcsr: 0x0000000000000000 fflags: 0x0000000000000000 frm: 0x0000000000000000
```

```
privilegeMode: 3
     a1 different at pc = 0x008000166c, right= 0x000000008000cf08, wrong = 0x000000008000cf10
     a3 different at pc = 0x008000166c, right= 0x000000000000029a, wrong = 0x0000000000000000
     a6 different at pc = 0x008000166c, right= 0x0000000000000002, wrong = 0x0000000000000001
Core 0: ABORT at pc = 0x80001694
Core-0 instrCnt = 1,264, cycleCnt = 12,758, IPC = 0.099075
Seed=0 Guest cycle spent: 12,761 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 7,061ms
```

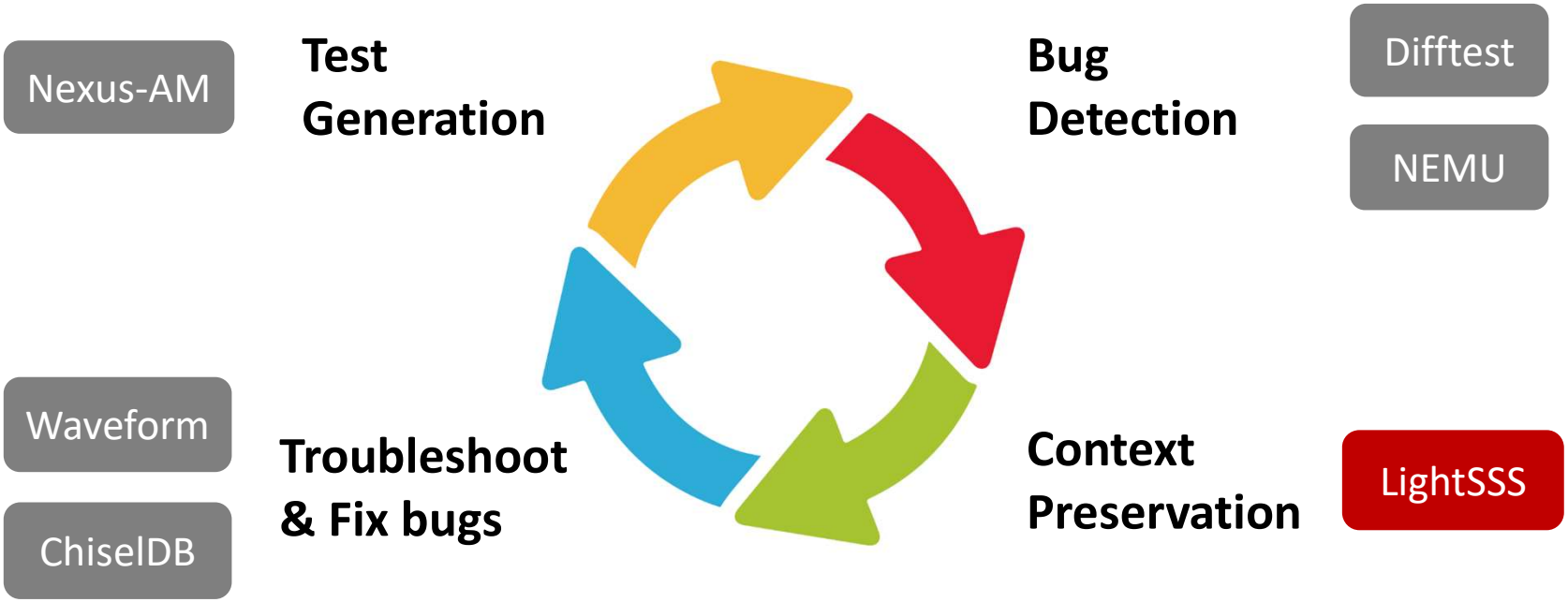
Difftest

- **Hands-on:** Dump waveform after Difftest detects the bug
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p1-basic-func**
- **Showcase**

```
$ bash run-emu-dumpwave.sh
$ ls $NOOP_HOME/build/*.vcd -lh   # There should be a .vcd waveform
```

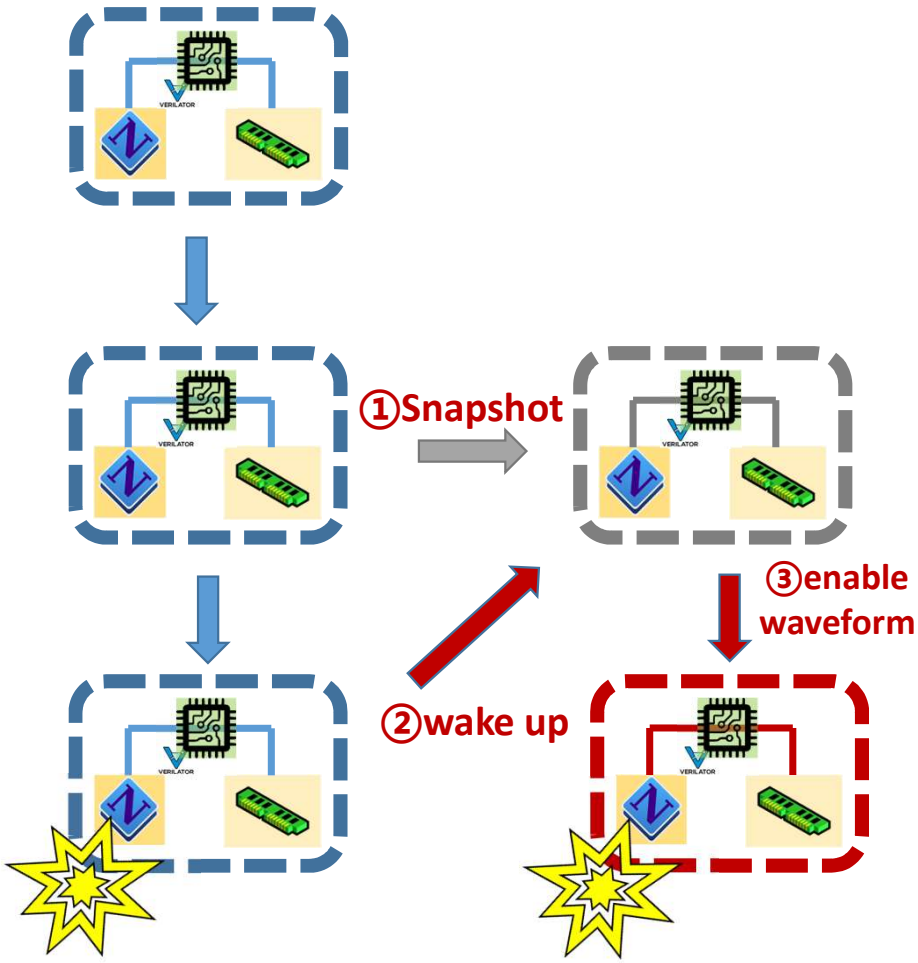
```
# ./emu-bug \
-i $AM_HOME/apps/coremark/build/coremark-1-iteration-riscv64-xs.bin \
--diff $NEMU_HOME/build/riscv64-nemu-interpreter-so \
-b 11000 \      the begin cycle of the wave, you may find XiangShan trap at 10000 cycles in previous step
-e 13000 \      the end cycle of the wave
--dump-wave \   set this option to dump wave, you will find *.vcd in $NOOP_HOME/build
```

MinJie Functional Verification



LightSSS: Light-weight Simulation SnapShot

- **Re-run simulation is time-consuming!**
  - **Snapshot is the way out**
- **LightSSS: snapshots of the process with fork()**
  - Copy-on-write on Linux Kernel
- **Advantage 1: Good portability and scalability**
  - Snapshots for any external models (such as C++)
  - No need to understand details of external models
- **Advantage 2: Low overhead of snapshots**
  - ~500us for taking a snapshot
  - Far less overhead than RTL snapshots by Verilator



LightSSS

- **Hands-on:** use lightSSS to dump waveform
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p1-basic-func**

```
$ bash run-emu-lightsss.sh
```

```
# ./emu-bug \
-i $AM_HOME/apps/coremark/build/coremark-1-iteration-riscv64-xs.bin \
--diff $NEMU_HOME/build/riscv64-nemu-interpreter-so \
--enable-fork \      No longer need to use complex parameters
2 > lightsss.err
```

LightSSS

- LightSSS is working if you see:

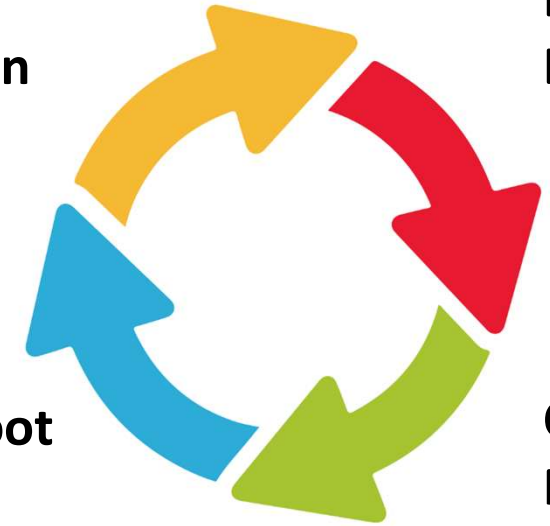
```
privilegeMode: 3
     a1 different at pc = 0x008000166c, right= 0x000000008000cf08, wrong = 0x000000008000cf10
     a3 different at pc = 0x008000166c, right= 0x000000000000029a, wrong = 0x0000000000000000
[FORK_INFO pid(1459543)] the oldest checkpoint start to dump wave and dump nemu log...
[FORK_INFO pid(1459543)] dump wave to /nfs/home/mayuexiao/tutorial/xs-env/XiangShan/build/2025-02-21@17:09:13_1.vcd...
%Warning: previous dump at t=0, requesting t=0, dump call ignored
The first instruction of core 0 has commited. Difftest enabled.
Running CoreMark for 1 iterations

==============  In the last commit group  ==============
the first commit instr pc of DUT is 0x000000008000166c
the first commit instr pc of REF is 0x000000008000166c

============== Commit Group Trace (Core 0) ==============
commit group [00]: pc 0080001654 cmtcnt 9
commit group [01]: pc 008000166a cmtcnt 6
commit group [02]: pc 0080001a6e cmtcnt 4
```

- After that, simulation will start again from the existing latest snapshot

MinJie Functional Verification



Nexus-AM

Test Generation

Bug Detection

Difftest

NEMU

Waveform

ChiselDB

Troubleshoot & Fix bugs

Context Preservation

LightSSS

ChiselDB: Debug-friendly Structured Database

- **Motivation:**
  - Waveforms are **large** in size and **hard to apply** further analysis
  - Need to analyze structured data like memory transaction trace
- **We present ChiselDB**
- **Highlights:**
  - Inserting probes between module interfaces in hardware
  - DPI-C: Using C++ function in Chisel code to transfer data
  - Persist in database, SQL queries supported

ChiselDB

- **Design source code:** *XiangShan/utility/src/main/scala/utility/ChiselDB.scala*
- **Usage:** Create table

```
// API: def createTable[T <: Record](tableName: String, hw: T): Table[T]
import huancun.utils.ChiselDB

class MyBundle extends Bundle {
       val fieldA = UInt(10.W)
       val fieldB = UInt(20.W)
}
val table = ChiselDB.createTable("MyTableName", new MyBundle)
```

ChiselDB

- **Usage:** Register probes

```
/* APIs
def log(data: T, en: Bool, site: String = "", clock: Clock, reset: Reset)
def log(data: Valid[T], site: String, clock: Clock, reset: Reset): Unit
def log(data: DecoupledIO[T], site: String, clock: Clock, reset: Reset): Unit
*/
table.log(
  data = my_data /* hardware of type T */,
  en = my_cond,   site = "MyCallSite",
  clock = clock,   reset = reset
)
```

ChiselDB

- **Example:** *XiangShan/src/main/scala/xiangshan/cache/mmu/L2TLB.scala*

```
class L2TlbMissQueueDB(implicit p: Parameters) extends TlbBundle {
   val vpn = UInt(vpnLen.W)
}

val L2TlbMissQueueInDB, L2TlbMissQueueOutDB = Wire(new L2TlbMissQueueDB)
L2TlbMissQueueInDB.vpn := missQueue.io.in.bits.vpn
L2TlbMissQueueOutDB.vpn := missQueue.io.out.bits.vpn

val L2TlbMissQueueTable = ChiselDB.createTable(
        "L2TlbMissQueue_hart" + p(XSCoreParamsKey).HartId.toString, new L2TlbMissQueueDB)

L2TlbMissQueueTable.log(L2TlbMissQueueInDB, missQueue.io.in.fire, "L2TlbMissQueueIn", clock, reset)
L2TlbMissQueueTable.log(L2TlbMissQueueOutDB, missQueue.io.out.fire, "L2TlbMissQueueOut", clock, reset)
```

ChiselDB

- **TL-Log： Persistence Transactions/Database**
  - Record bus transactions to SQLite3 database using ChiselDB
  - Support offline bug detect and performance analysis

Time	Name	Channel	Opcode	Permission	State	SourceID	SinkID	Address	Data
116854134	L2_L1I_0	A	AcquireBlock	Grow	NtoB	1	0	803d3680	0000000000000000 0000000000000000 0000000000000000 0000000000000000
116854146	L2_L1I_0	D	GrantData	Cap	toT	1	16	803d3680	fa8437839043903 06090063843c23 84382300093783 378300093023c3bd
116854147	L2_L1I_0	D	GrantData	Cap	toT	1	16	803d3680	03e32e07bb030089 f0ef8526c881f49b b60300893783bbdf 855a010925832e07
123191840	L2_L1I_0	C	ReleaseData	Shrink	TtON	0	0	803d3680	fa8437839043903 06090063843c23 84382300093783 378300093023c3bd
123191841	L2_L1I_0	B	Probe	Cap	toN	0	0	803d3680	0000000000000000 0000000000000000 0000000000000000 0000000000000000
123191841	L2_L1I_0	C	ReleaseData	Shrink	TtoN	0	0	803d3680	03e32e07bb030089 f0ef8526c881f49b b60300893783bbdf 855a010925832e07
123191848	L2_L1I_0	D	ReleaseAck	Cap	toT	0	31	803d3680	0000000020ab05b 0000000020fab45b 0000000020fab85b 0000000020fabc5b
123191851	L3_L2_0	C	ReleaseData	Shrink	TtoN	31	0	803d3680	fa8437839043903 06090063843c23 84382300093783 378300093023c3bd
123191852	L3_L2_0	C	ReleaseData	Shrink	TtoN	31	0	803d3680	03e32e07bb030089 f0ef8526c881f49b b60300893783bbdf 855a010925832e07
123191862	L3_L2_0	D	ReleaseAck	Cap	toT	31	16	803d3680	86a6f7043583dcd5 f0ef856a865a8762 3583b7654481e17 865a86a68762f704
123191863	L2_L1I_0	C	ProbeAck	Shrink	TtoN	0	0	803d3680	0000000000000000 0000000000000000 0000000000000000 0000000000000000
123191878	L3_L2_0	C	Release	Report	NtoN	30	0	803d3680	5597bf494981aa09 8522cc2585930000 f84ef15dda0f00ef 001947036775e84a

Example: requests sequence in database that violate cache coherence

ChiselDB

- **Hands-on: Analysis of Cache Coherence Violation**
  - **We'll switch to this DIR: ~/$YOUR_NAME/xs-env/tutorial/p2-chiseldb**

```
# Inject bug – We set all Release Data (L2->L3) as a constant value
$ cd ../p2-chiseldb && cat cdb_err.patch
```

```
--- a/src/main/scala/huancun/noninclusive/SinkC.scala
+++ b/src/main/scala/huancun/noninclusive/SinkC.scala

@@ -99,7 +99,7 @@ class SinkC(implicit p: Parameters) extends
BaseSinkC {
-       buffer(insertIdx)(count) := c.bits.data
+       buffer(insertIdx)(count) := 0xABCDEF.U
```

ChiselDB

- **Hands-on: Analysis of Cache Coherence Violation**
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p2-chiseldb**

```
# simulate using pre-built emu (ChiselDB enabled)
$ bash chiseldb_step1_run.sh
```

```
# ./emu-cdb-err -i $NOOP_HOME/ready-to-run/linux.bin \
  --diff $NOOP_HOME/ready-to-run/riscv64-nemu-interpreter-so \
  --dump-db    # set this argument to dump data base
  2>linux.err
```

ChiselDB

- **Hands-on: Analysis of Cache Coherence Violation**
- **Difftest Info**
  - Error Address：0x80040580

```
The first instruction of core 0 has commited. Difftest enabled.
cacheid=1,idtfr=0,realpaddr=0x80040580: Refill test failed!
addr: 80040580
Gold: 000000008000447e00000000800044aa000000008000ce3200000000800 0ce1000000000800043f400000000800044120000000080041900000000800419e0
Core: 000000000abcdef000000000000000000000000000000000000000000000000abcdef000000000000000000000000000000000000000000000000
Core 0: ABORT at pc = 0x8000474e
Core-0 instrCnt = 9,128, cycleCnt = 12,539, IPC = 0.727969
```

ChiselDB

- **Hands-on: Analysis of Cache Coherence Violation**
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p2-chiseldb**

```
# Analyze
$ bash chiseldb_step2_analyze.sh
```

```
# 1. sqlite query for all transactions on Error address
# 2. use script to parse TLLog
# sqlite3 $NOOP_HOME/build/*.db \
  "select * from TLLOG where ADDRESS=0x80040580" | \
  sh $NOOP_HOME/scripts/utils/parseTLLog.sh
```

ChiselDB:

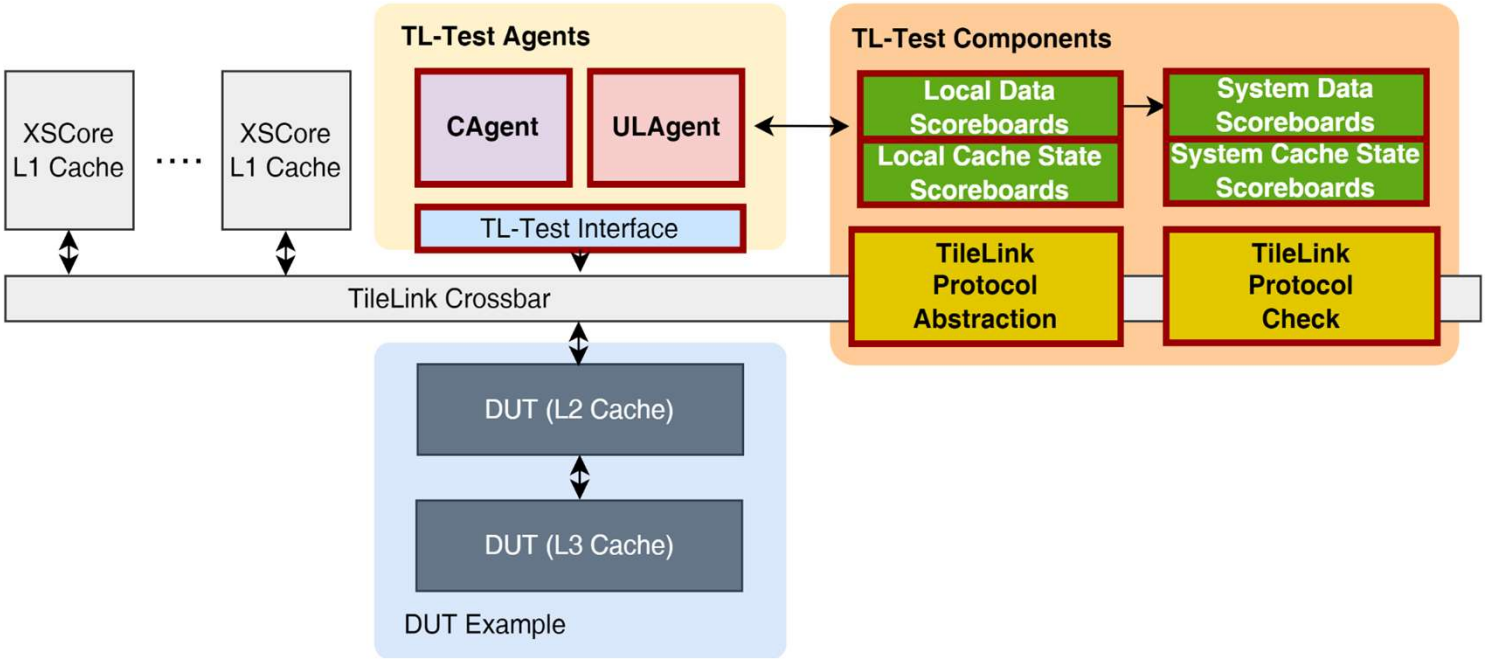
```
# Result: [Time | To_From | Channel | Opcode | Permission | Address | Data]

# Data successfully transferred from L1D to L2
11116 L2_L1D_0 C ReleaseData Shrink TtoN 80040580
0000000000000447e 0000000000044aa 000000008000ce32 000000008000ce10
# Data successfully transferred from L2 to L3
12081 L3_L2_0 C ProbeAckData Shrink TtoN 80040580
0000000000000447e 0000000000044aa 000000008000ce32 000000008000ce10


# But when L1D acquires Error addr again, the data loaded from L3 is wrong
12492 L2_L1D_0 A AcquireBlock Grow NtoB 80040580
12498 L3_L2_0 A AcquireBlock Grow NtoB 0 1 80040580
12511 L3_L2_0 D GrantData Cap toT 80040580
0000000000abcdef 0000000000000000 0000000000000000 0000000000000000
# So, there must be something wrong when L3 records Release Data or L3 returns data
```

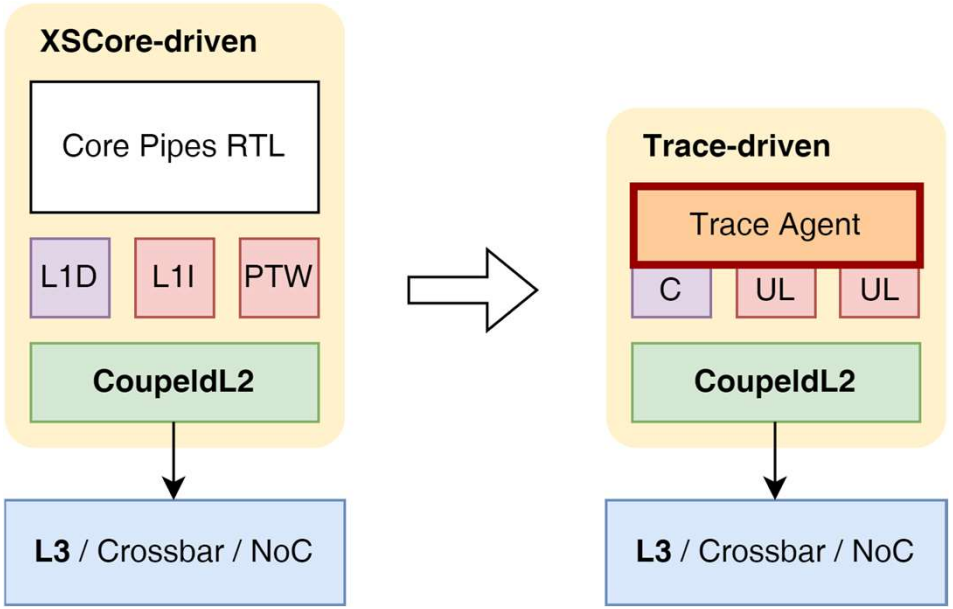
TL-Test: Cache Verification Framework

- **As an infrastructure for cache verification**
  - Support **Tilelink protocol** and **cache coherence** check
  - **Fast test cases generation** and **constrained random test**



TL-Test

- **TL-Trace: Trace-based testbench generation**
  - Convert cache access trace into testbench
  - Fast reproduction of functional bugs and analysis of performance problems



TL-Test

```
# Project Structure
$ cd $TLT_HOME && tree -d -L 1
.
├── assets
├── configs
├── dut
├── main
├── run
└── scripts


# Enter TL-Test tutorial
$ cd ../tutorial/p3-tl-test
```

TL-Test

- **Hands-on: Analysis of Cache Coherence Violation**
  - **You should be here: ~/$YOUR_NAME/xs-env/tutorial/p3-tl-test**

```
# Inject bug – We wrongly shift the Grant Data (L2->L1)
$ cat tlt_err.patch
```

```
--- a/tl-test-new/dut/CoupledL2/src/main/scala/coupledL2/GrantBuffer.scala
+++ b/tl-test-new/dut/CoupledL2/src/main/scala/coupledL2/GrantBuffer.scala
@@ -94,7 +94,7 @@ class GrantBuffer(implicit p: Parameters) extends
LazyModule {
-       d.data := data
+       d.data := data << 8
```

TL-Test

- **Hands-on: Analysis of Cache Coherence Violation**

```
# We provide a pre-compiled TL-Test
# Run the TL-Test
$ bash tltest_step1_run.sh
```

```
# cd $TLT_HOME/run && ./tltest_v3lt 2>&1 | tee tltest_v3lt.log
```

TL-Test

- **Hands-on: Analysis of Cache Coherence Violation**
- TL-Test Info
  - Error Address： 0x4000

```
[2940] [tl-test-new-INFO] #0 L2[0].C[0] [data complete D] [GrantData toT] source: 0, addr: 0x4000, alias: 0, data: [ 00 ff ... 00 70 ... ]

dut: [ 00 ff ... 00 70 ... ]
ref: [ ff e6 ... 70 f9 ... ]
[2940] [tl-test-new-ERROR] [tlc_assert failure at int GlobalBoard<unsigned long>::data_check(TLLocalContext *, const uint8_t *, const uint8_t *,
 std::string) [T = unsigned long]:277]
[2940] [tl-test-new-ERROR] [tlc_assert failure from system #0]
[2940] [tl-test-new-ERROR] info: Data mismatch from status SB_VALID!
```

TL-Test

- **Hands-on: Analysis of Cache Coherence Violation**

```
# Analyze
$ bash tltest_step2_analyze.sh
```

```
# 1. grep all transactions on Eaddr(0x4000)
# 2. use TL-Log to help debugging

# grep “addr: 0x4000” $TLT_HOME/run/tltest_v3lt.log
```

TL-Test

 # Result: [Time | Core | Channel | Opcode | Source | Address | Data]

 # L1D acquires Error addr

[1954] L2[0].C[0] [fire A] [AcquireBlock NtoB] source: 0, addr: 0x4000

 # L2 probes L1D, data successfully transferred from L1D to L2

[2190] L2[0].C[0] [fire B] [ProbeBlock toN] source: 0, addr: 0x4000
[2192] L2[0].C[0] [fire C] [ProbeAckData TtoN] source: 0x10, addr: 0x4000, data: [ff e6 ...]
[2194] L2[0].C[0] [fire C] [ProbeAckData TtoN] source: 0x10, addr: 0x4000, data: [70 f9 ...]

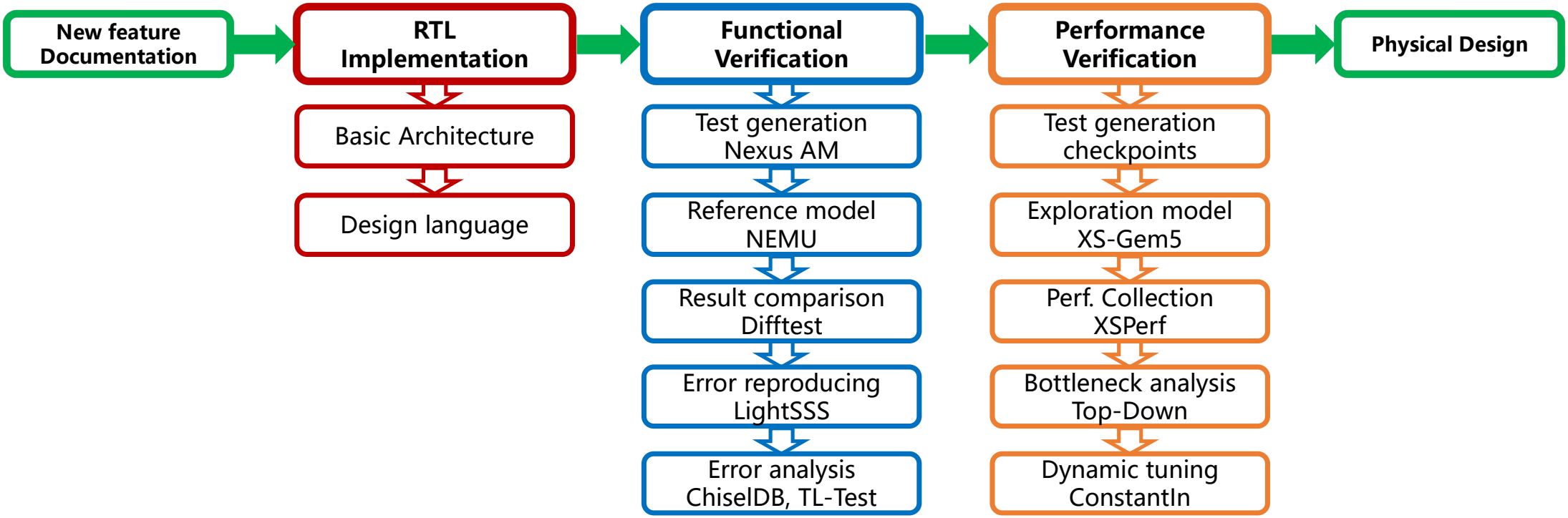
 # But when L2 grants data of Error addr, data loaded from L2 is wrong

[2938] L2[0].C[0] [fire D] [GrantData toT] source: 0x1, addr: 0x16000, data: [00 ff...]

[2940] L2[0].C[0] [fire D] [GrantData toT] source: 0x1, addr: 0x16000, data: [00 70...]

 # So there must be something wrong when L2 Grant Data

Summary: MinJie Development Flows and Tools



北京开源芯片研究院
BEIJING INSTITUTE OF OPEN SOURCE CHIP

中国开放指令生态（RISC-V）联盟
China RISC-V Alliance

香山开源处理器社区
XiangShan Open-Source Processor Community

Thanks!